

Troll Farm Bot

Source Reference

CodinGame Spring Challenge 2026 — Gold league

Version 1.54.0-frontdoor — generated July 9, 2026

What this is

The **Troll Farm** bot for the CodinGame Spring Challenge 2026 (2-player, simultaneous-move resource game: harvest fruit and fell trees for wood; final score is `fruit + 4*wood`). This document is a syntax-highlighted reference of the live bot's Rust source.

Architecture — a three-layer activity manager

Each turn, `decide_elite` runs three layers over the parsed `State`:

- **L1 tactics** (`tactics.rs`) builds a `Plan`: which phase we are in, farm geometry (map-distance `farm_d`, the front-door farm anchor), and the tuning it exposes to L2.
- **L2 planner** (`planner.rs`) is the *task manager*. `candidates()` *produces* each troll's viable tasks as `(target, value)` candidates, where the old sequential-cascade branch hierarchy becomes value **bands** (spaced $\gg$ any ETA, so priority never inverts; ETA differentiates within a band). `assign_resolved()` *matches* tasks to trolls jointly — exhaustive over per-troll top-K, maximizing total value, forbidding conflicting claims, with a canonical tie-break. Refinements: a `race()` check (skip trees an enemy fells first), STICKY targets (anti-flap), and a task-interference yield pass.
- **L3 motion** (`motion.rs`) jointly chooses this turn's landing cells (swaps and chains through moving teammates), **shuffle-invariant**: the plan depends on the objective, never on iteration order.

The design principle throughout is that “a troll misbehaving” is really the task manager producing the wrong task *set* — too few, mis-valued, or mis-located — so fixes target task production and valuation, not per-troll behavior.

Deployment

The modules below live as a library so tests and a black-box equality harness can reach them; a bundler (`tools/bundle.py`) re-inlines them into the single file CodinGame accepts, then a minifier keeps it under the 100 KB limit.

Contents

1	botmain.rs	3
2	botmain/state.rs	9
3	botmain/tactics.rs	12
4	botmain/planner.rs	18
5	botmain/motion.rs	32
6	botmain/ownership.rs	36

1 botmain.rs

Entry point — CG protocol I/O parsing, the per-turn decide loop, and all tuning constants.

(476 lines)

```
1  #![allow(dead_code, unused)]
2  // CodinGame Spring Challenge 2026 - Troll Farm bot (Rust port of Python v0.7.1)
3  // Single-file submission. stdlib only.
4
5  use std::cell::RefCell;
6  use std::collections::{HashMap, HashSet, VecDeque};
7  use std::io::{self, BufRead, Write};
8
9  // — constants —————
10
11 const VERSION: &str = "1.54.0-frontdoor"; // front-door farm placement: chokepoint-gated fix for the shack-hub farm_d bug (Sasso long-path
   ↪ re-diagnosis)
12                                     // (the sequential cascade jobs.rs was REMOVED for submission size – 100 KB cap; it lives in
13                                     // git history and in the frozen v1.26.0 artifacts for instant fallback)
14
15 mod state;
16 pub use state::*;
17 pub mod motion;
18 pub mod ownership;
19 pub mod planner;
20 pub mod tactics;
21 // Flip to true for a SIM-FIDELITY validation run: echoes the full per-turn state
22 // to stderr (captured in the replay) so we can replay a real game through the sim
23 // and compare turn-by-turn. Off by default (no effect on play or stdout parity).
24 const DEBUG: bool = false;
25
26 // — WOOD-RACE bot (v1.0) – beats the Silver Boss ~68% in the local sim —————
27 // Mirror of strategies:mybot (validated in the referee-faithful Rust sim). Strategy:
28 // * GREEDY expansion to ~4 trolls (train the cheapest affordable troll each turn),
29 //   jumping the queue to build TWO fast (ms2,cc2,chop2) choppers as soon as afford-
30 //   able; the rest are speed harvesters. Mine iron to fund the choppers' chop cost.
31 // * Choppers fell the best tree (close + big) with a DENIAL bias toward the foe's
32 //   shack -- felling starves the opponent's fruit while banking 4pt-each wood.
33 // * Harvesters grab the NEAREST ripe fruit (max throughput) and seed a tiny base
34 //   plum orchard; everyone banks when full.
35 // The boss is a similar wood/denial bot; our edge is faster (ms2) choppers that win
36 // the race to contested trees + higher fruit throughput (nearest-ripe harvesting).
37 // Cheap pure chopper (ms1, cc2, hp0, chop2): swept best vs silver_boss at 87% (vs the
38 // old ms2 (2,2,1,2) at 81%). cc2 = 2 wood/fell is essential; dropping ms+hp saves plum
39 // +apple for a stronger economy while still winning the denial race (DW=3) + woodfarm.
40 // hp0 (was hpl): saves n+1 APPLE per chopper; the only loss is a rarely-reachable
41 // fruit-harvest fallback. Confirmed on BOTH boss models at 1000 seeds (2026-07-02):
42 // scriptboss 59.8~60.9% (margin +14.7~+18.2), silverboss 77.5~78.4% (+24.1~+26.9).
43 const MB_CHOPPER: (i32, i32, i32, i32) = (2, 3, 0, 3); // v1.12.0: cc3/chop3 SUPER-chopper (fell fast, bank every 3) – the nmahoude
   ↪ throughput lever
44 const MB_NCHOPPERS: i32 = 1; // ONE super-chopper; the other trolls are HARVESTERS that fund it + feed the farm
45                                     // chop1 harvesters (+n+1 iron each): every fruit troll can also FELL the base
46                                     // farm's young bananas (the "mower"). Blueprint from arena replays: 250-pt bots
47                                     // sustain 0.30 wood/turn vs our 0.07; fellers must live AT the farm, and the
48                                     // denial choppers can't. Both-model win at 1000 seeds: scriptboss 63.0->64.3%
49                                     // (margin +25.8->+31.6), silverboss 85.1->87.5% (+51.4->+54.1); wood 90->105.
50 const MB_HARVESTERS: [(i32, i32, i32, i32); 3] = [(2, 2, 2, 0), (1, 2, 2, 0), (1, 1, 1, 0)];
51 const MB_MAX_TROLLS: usize = 4;
52 const MB_MAX_ORCHARD: usize = 2;
53 const MB_MIN_TURNS_LEFT: i32 = 20;
54 // Denial-heavy chopper targeting (swept 2026-07-01 in the faithful sim): DW=3, WT=0
55 // lifts the bot from 67.6% -> 78.0% vs silver_boss. Our cheap fast choppers win the
56 // race to the BOSS's trees and starve its wood+fruit; biasing hard toward the enemy
57 // shack (and dropping the tree-size preference) is decisively better than balanced.
58 const MB_DENIAL_W: i32 = 0;
59 const MB_SIZE_W: i32 = 0;
60
61 thread_local! {
62     // xorshift RNG for the watchdog sidestep – FIXED seed: fully deterministic
63     // (sequence position is state-driven). Survivor of the RHEA machinery cut.
64     static RH_RNG: RefCell<u64> = RefCell::new(0x9E3779B97F4A7C15);
65 }
66
67 fn rh_rand() -> u64 {
68     RH_RNG.with(|rng| {
69         let mut r = rng.borrow_mut();
70         *r ^= *r << 13;
71         *r ^= *r >> 7;
72         *r ^= *r << 17;
73         *r
74     })
75 }
76
77 // (decide_sched, the RHEA fast engine + searcher, and the v1.0.x legacy deciders were
78 // REMOVED 2026-07-06 for the 100 KB submission cap – run() calls decide_elite only.
79 // Full history: git; frozen artifacts: cgauto/submissions/.)
80
81 // B1/B2: the meta selector consumed by tactics::phase_for. Tempo is the live meta
```

```

81 // (phase-inert: phase_for(Tempo, _) == Phase::Tempo always, so every phase-gated band in
82 // planner.rs/tactics.rs is a no-op) - Tempo is equality-proven byte-identical to the
83 // pre-phase champion. Scale (Hoard-Factory at T_SWITCH) now has real Hoard-phase behavior
84 // (fell suppression + denial exception + wallet band + training ladder) but is still not
85 // selected live. See rust/src/botmain/tactics.rs and planner.rs.
86 const GE_META: tactics::Meta = tactics::Meta::Tempo;
87 const GE_SPEC: (i32, i32, i32, i32) = (2, 3, 0, 2); // cc=3 chopper (Boss-5 mechanism: capture 3 wood/size-3 tree)
88 const GE_MAX_TROLLS: i32 = 2; // T-hand parked pending a better design; re-arm by setting 3
89 const GE_FEEDER_SPEC: (i32, i32, i32, i32) = (1, 1, 1, 0); // cheap hands: 3 plum/3 lemon/3 apple at n=2 (half the old feeder price)
90 const GE_FEEDER_T: i32 = 45; // T-hand: restored from 60 - 60 was a leftover from the v1.28.x farm-death era when GE_MAX_TROLLS=2 made this
    ↪ gate unreachable anyway (dormant 3rd hand); the funding fix (planner.rs ladder_funding) is what actually treats farm-death now, so the
    ↪ feeder can arm this early again
91 const GE_FEEDER_FARM: usize = 0; // T-hand.2: 1->0 - verdict-#2 catch-22: the hand rescues the dead farm; any farm precondition blocks the
    ↪ cure exactly when it's needed (fruit/iron wallets, need_fund/need_iron, are the real gates now). farm_now collapsed to literal 0 for
    ↪ 63-100% of sampled turns per game (8/8 boss games ended farm=0); one game had fruit+iron sufficient for 255 straight turns while farm
    ↪ sat at 0 the whole time and want_feeder still never became eligible under the old >=1 floor.
92 const GE_CHOP_DELAY: i32 = 0; // NO delay: train chopper early (denial > accumulation, proven 2026-07-05)
93 const GE_CHOP_FARM: usize = 3; // train as soon as affordable (early aggression, v1.4.5 regime)
94 const GE_FARM_R: i32 = 2; // v1.13.0: TIGHT farm hugging the shack - halves the chopper's bank-trip distance (the throughput bottleneck)
95 const GE_FARM_MAX: usize = 12; // v1.19.0: fill the radius-2 area (~12 cells) - more trees maturing in parallel = chopper idles less
96 const GE_FELL_SIZE: i32 = 2; // NATIVE/contested trees: fell at size 2 = DENIAL (grab before opponent)
97 const GE_CHOP_R: i32 = 5; // roam4 arena-REVERTED at -3.6 (2026-07-08); tree restored to champion semantics
98 const GE_LIQ_T: i32 = 34; // turns_rem <= this: fell anything reachable (A1 liq44 REJECTED by gatekeeper 2026-07-07)
99 const GE_STARTER_CHOP: bool = true; // let a chop-capable starter help fell
100 const GE_MIN_TURNS_LEFT: i32 = 20; // no training inside the last 20 turns
101 const GE_SEED_RESERVE: usize = 2; // protect K most-mature farm bananas as seed sources
102 const GE_FARM_FELL: i32 = 3; // OUR farm bananas: fell at size 3 = PRODUCTION (cc=3 captures all 3)
103 // v1.53.0-pressurefarm (Task 2 Step 1): under Yellow+ observed pressure, tactics::plan_impl
104 // clamps farm_cap down to this floor instead of GE_FARM_MAX - a small "keep some farm alive"
105 // bootstrap floor, not zero (an empty farm was the seedloop/fruitbank-era failure mode). A
106 // farm already below the floor keeps planting regardless of pressure (this is a CEILING that
107 // only ever shrinks room to expand further, never a mandate to shrink below where we are).
108 const GE_PRESSURE_FARM_FLOOR: usize = 4;
109
110 /// v1.4.0 live decider: the gold-elite pure-production strategy. The standalone
111 /// bot is always player 0 (my_trolls). A 1:1 port of GoldElite::decide with an
112 /// added turn-1 MSG and an anti-stall watchdog (below).
113 fn decide_elite(state: &State) -> Vec<String> {
114     if state.turn == 1 {
115         motion::reset();
116         ownership::reset();
117         tactics::reset();
118         planner::reset();
119     }
120     let mut my: Vec<Troll> = state.my_trolls.clone();
121     my.sort_by_key(|t| t.id);
122
123     // L1: tactical plan -> L2: per-troll job assignment -> L3: motion post-pass
124     let plan = tactics::plan(state, &my);
125     let mut cmd_by_id = planner::assign_resolved(state, &plan, &my);
126     if DEBUG && state.turn % 5 == 0 {
127         eprintln!(
128             "@TFFARM t={} farm={} seeds={} n={} flaps={} phase={:?}",
129             state.turn,
130             plan.farm_now,
131             state.my_inventory[BANANA],
132             my.len(),
133             planner::flaps(),
134             plan.phase
135         );
136     }
137     if DEBUG {
138         ownership::log(state, &plan);
139         ownership::log_pressure(state, &plan);
140     }
141
142     // R6a/R6b feedback: planner::assign_resolved runs joint assignment, the first
143     // motion solve, one bounded yield-to-urgent pass, and final MOVE landing pinning.
144     // anti-stall watchdog (R3b: motion layer) - sidestep trolls self-blocked 2+ turns
145     motion::watchdog(state, &my, &mut cmd_by_id);
146
147     let mut actions: Vec<String> = Vec::new();
148     if state.turn == 1 {
149         actions.push(format!("MSG v{}", VERSION));
150     }
151     let mut ids: Vec<i32> = cmd_by_id.keys().copied().collect();
152     ids.sort();
153     for id in ids {
154         actions.push(cmd_by_id[&id].clone());
155     }
156
157     if plan.train_now
158         && TOTAL_TURNS - state.turn > GE_MIN_TURNS_LEFT
159         && !my.iter().any(|u| u.pos() == plan.shack)
160     {
161         actions.push(format!(
162             "TRAIN {} {} {} {}",
163             plan.train_spec.0, plan.train_spec.1, plan.train_spec.2, plan.train_spec.3

```

```

164     ));
165 }
166
167 if actions.is_empty() {
168     actions.push("WAIT".into());
169 }
170 actions
171 }
172
173 // — I/O parsing —————
174
175 fn parse_grid(grid_lines: &[String]) -> (HashSet<Cell>, Cell, Cell, HashSet<Cell>, HashSet<Cell>) {
176     let mut walkable = HashSet::new();
177     let mut iron = HashSet::new();
178     let mut water = HashSet::new();
179     let mut my_shack = (0i32, 0i32);
180     let mut opp_shack = (0i32, 0i32);
181     for (y, line) in grid_lines.iter().enumerate() {
182         for (x, ch) in line.chars().enumerate() {
183             let cell = (x as i32, y as i32);
184             match ch {
185                 '0' => my_shack = cell,
186                 '1' => opp_shack = cell,
187                 '.' => {
188                     walkable.insert(cell);
189                 }
190                 '+' => {
191                     iron.insert(cell);
192                 }
193                 '~' => {
194                     water.insert(cell);
195                 }
196                 _ => {} // '#' and others are rocks
197             }
198         }
199     }
200     (walkable, my_shack, opp_shack, iron, water)
201 }
202
203 fn read_line(reader: &mut impl BufRead) -> Option<String> {
204     let mut s = String::new();
205     match reader.read_line(&mut s) {
206         Ok(0) => None,
207         Ok(_) => Some(s.trim_end_matches('\n').trim_end_matches('\r').to_string()),
208         Err(_) => None,
209     }
210 }
211
212 fn parse_turn(
213     reader: &mut impl BufRead,
214     walkable: &HashSet<Cell>,
215     my_shack: Cell,
216     opp_shack: Cell,
217     turn: i32,
218     iron_cells: &HashSet<Cell>,
219     water_cells: &HashSet<Cell>,
220 ) -> Option<State> {
221     let inv0_line = read_line(reader)?;
222     let my_inventory: Vec<i32> = inv0_line
223         .split_whitespace()
224         .map(|v| v.parse().unwrap())
225         .collect();
226     let inv1_line = read_line(reader)?;
227     let opp_inventory: Vec<i32> = inv1_line
228         .split_whitespace()
229         .map(|v| v.parse().unwrap())
230         .collect();
231
232     let tree_count_line = read_line(reader)?;
233     let tree_count: usize = tree_count_line.trim().parse().unwrap();
234     let mut trees = Vec::with_capacity(tree_count);
235     for _ in 0..tree_count {
236         let line = read_line(reader)?;
237         let parts: Vec<&str> = line.split_whitespace().collect();
238         trees.push(Tree {
239             tree_type: parts[0].to_string(),
240             x: parts[1].parse().unwrap(),
241             y: parts[2].parse().unwrap(),
242             size: parts[3].parse().unwrap(),
243             health: parts[4].parse().unwrap(),
244             fruits: parts[5].parse().unwrap(),
245             cooldown: parts[6].parse().unwrap(),
246         });
247     }
248
249     let troll_count_line = read_line(reader)?;
250     let troll_count: usize = troll_count_line.trim().parse().unwrap();
251     let mut my_trolls = Vec::new();

```

```

252 let mut opp_trolls = Vec::new();
253 for _ in 0..troll_count {
254     let line = read_line(reader)?;
255     let f: Vec<i32> = line
256         .split_whitespace()
257         .map(|v| v.parse()).unwrap()
258         .collect();
259     // id player x y ms cc hp chop carry[6]
260     let troll = Troll {
261         id: f[0],
262         x: f[2],
263         y: f[3],
264         movement_speed: f[4],
265         carry_capacity: f[5],
266         harvest_power: f[6],
267         chop_power: f[7],
268         carry: [f[8], f[9], f[10], f[11], f[12], f[13]],
269     };
270     if f[1] == 0 {
271         my_trolls.push(troll);
272     } else {
273         opp_trolls.push(troll);
274     }
275 }
276
277 let my_inv: [i32; 6] = [
278     my_inventory[0],
279     my_inventory[1],
280     my_inventory[2],
281     my_inventory[3],
282     my_inventory[4],
283     my_inventory[5],
284 ];
285 let opp_inv: [i32; 6] = [
286     opp_inventory[0],
287     opp_inventory[1],
288     opp_inventory[2],
289     opp_inventory[3],
290     opp_inventory[4],
291     opp_inventory[5],
292 ];
293
294 Some(State {
295     walkable: walkable.clone(),
296     my_shack,
297     opp_shack,
298     my_inventory: my_inv,
299     opp_inventory: opp_inv,
300     trees,
301     my_trolls,
302     opp_trolls,
303     turn,
304     iron_cells: iron_cells.clone(),
305     water_cells: water_cells.clone(),
306 })
307 }
308
309 // — main —————
310
311 /// Echo per-turn state to stderr for sim validation (gated by DEBUG). At turn 1
312 /// it logs the map + full initial trees/trolls (to reconstruct the start); every
313 /// turn it logs a compact digest (both inventories + all troll positions) so a
314 /// captured game can be replayed through the sim and compared turn-by-turn.
315 fn debug_log(state: &State, grid: &[String], width: i32, height: i32) {
316     if !DEBUG {
317         return;
318     }
319     if state.turn == 1 {
320         eprintln!("@TFMAP {} {} ", width, height);
321         for l in grid {
322             eprintln!("@TFMAP {} ", l.trim_end());
323         }
324         for t in &state.trees {
325             eprintln!(
326                 "@TFI P {} {} {} {} {} {} {} {} ",
327                 t.tree_type, t.x, t.y, t.size, t.health, t.fruits, t.cooldown
328             );
329         }
330         for (pl, list) in [(0, &state.my_trolls), (1, &state.opp_trolls)] {
331             for u in list {
332                 eprintln!(
333                     "@TFI U {} {} {} {} {} {} {} {} {} {} {} {} {} {} ",
334                     u.id,
335                     pl,
336                     u.x,
337                     u.y,
338                     u.movement_speed,
339                     u.carry_capacity,

```

```

340         u.harvest_power,
341         u.chop_power,
342         u.carry[0],
343         u.carry[1],
344         u.carry[2],
345         u.carry[3],
346         u.carry[4],
347         u.carry[5]
348     );
349 }
350 }
351 }
352 let join = |a: &[i32; 6]| {
353     a.iter()
354         .map(|v| v.to_string())
355         .collect::()
356         .join(",")
357 };
358 let mut us = String::new();
359 for u in &state.my_trolls {
360     us.push_str(&format!("{}",0,{},{},", u.id, u.x, u.y));
361 }
362 for u in &state.opp_trolls {
363     us.push_str(&format!("{}",1,{},{},", u.id, u.x, u.y));
364 }
365 eprintln!(
366     "@TFD {} {} {} {}",
367     state.turn,
368     join(&state.my_inventory),
369     join(&state.opp_inventory),
370     us
371 );
372
373 // Compact per-turn SUMMARY (printed LAST so it's the console line that survives
374 // truncation): both scores, tree count, and OPPONENT troll stats -- so we can read
375 // the real Boss 4's composition (fruit vs wood) and troll build from one screenshot.
376 let score = |inv: &[i32; 6]| inv[0] + inv[1] + inv[2] + inv[3] + 4 * inv[5];
377 let opp_builds: Vec<String> = state
378     .opp_trolls
379     .iter()
380     .map(|u| {
381         format!(
382             "{}:{}.{}.{}.{}",
383             u.id, u.movement_speed, u.carry_capacity, u.harvest_power, u.chop_power
384         )
385     })
386     .collect();
387 let my_builds: Vec<String> = state
388     .my_trolls
389     .iter()
390     .map(|u| {
391         format!(
392             "{}:{}.{}.{}.{}",
393             u.id, u.movement_speed, u.carry_capacity, u.harvest_power, u.chop_power
394         )
395     })
396     .collect();
397 eprintln!(
398     "@TFSUM t={} me={} opp={} trees={} myinv={} oppinv={} mybuilds={} oppbuilds={}",
399     state.turn,
400     score(&state.my_inventory),
401     score(&state.opp_inventory),
402     state.trees.len(),
403     join(&state.my_inventory),
404     join(&state.opp_inventory),
405     my_builds.join(","),
406     opp_builds.join(",")
407 );
408 }
409
410 pub fn run() {
411     let stdin = io::stdin();
412     let stdout = io::stdout();
413     let mut reader = io::BufReader::new(stdin.lock());
414     let mut out = io::BufWriter::new(stdout.lock());
415
416     // Read header: width height
417     let header = match read_line(&mut reader) {
418         Some(s) => s,
419         None => return,
420     };
421     let mut hw = header.split_whitespace();
422     let width: i32 = hw.next().unwrap().parse().unwrap();
423     let height: i32 = hw.next().unwrap().parse().unwrap();
424
425     let mut grid_lines = Vec::with_capacity(height as usize);
426     for _ in 0..height {
427         match read_line(&mut reader) {

```

```

428         Some(line) => grid_lines.push(line),
429         None => return,
430     }
431 }
432
433 let (walkable, my_shack, opp_shack, iron_cells, water_cells) = parse_grid(&grid_lines);
434
435 let mut turn = 0i32;
436 loop {
437     turn += 1;
438     match parse_turn(
439         &mut reader,
440         &walkable,
441         my_shack,
442         opp_shack,
443         turn,
444         &iron_cells,
445         &water_cells,
446     ) {
447         None => break,
448         Some(state) => {
449             debug_log(&state, &grid_lines, width, height);
450             let cmds = decide_elite(&state);
451             // @TFMOVE: motion-rule instrument (motion_analyze.py) - positions BEFORE
452             // moving + intended MOVES; block rate = intended-but-didn't-advance.
453             if DEBUG {
454                 let pos: Vec<String> = state
455                     .my_trolls
456                     .iter()
457                     .map(|t| format!("{}",t.id, t.x, t.y))
458                     .collect();
459                 let moves: Vec<String> = cmds
460                     .iter()
461                     .filter(|c| c.starts_with("MOVE "))
462                     .cloned()
463                     .collect();
464                 eprintln!(
465                     "@TFMOVE t={} pos={} moves={}",
466                     state.turn,
467                     pos.join(" "),
468                     moves.join(" ")
469                 );
470             }
471             writeln!(out, "{}", cmds.join(";")).unwrap();
472             out.flush().unwrap();
473         }
474     }
475 }
476 }

```


2 botmain/state.rs

Core types (State, Troll, Tree) + shared helpers (BFS distances, tie-break salts).
(212 lines)

```
1  /// State layer (R3a): game-state types, item indices, and pure helpers shared by
2  /// every decider. Extracted VERBATIM from botmain.rs (only visibility added);
3  /// behavior equality is enforced by the black-box harness (src/bin/equality.rs).
4  use std::collections::{HashMap, HashSet, VecDeque};
5
6  pub const TOTAL_TURNS: i32 = 300;
7
8  // Item indices: PLUM=0, LEMON=1, APPLE=2, BANANA=3, IRON=4, WOOD=5
9  pub const PLUM: usize = 0;
10 pub const LEMON: usize = 1;
11 pub const APPLE: usize = 2;
12 pub const BANANA: usize = 3;
13 pub const IRON: usize = 4;
14 pub const WOOD: usize = 5;
15
16 // Base growth cooldown per tree type
17 pub fn plant_cooldown(t: &str) -> i32 {
18     match t {
19         "PLUM" => 8,
20         "LEMON" => 8,
21         "APPLE" => 9,
22         "BANANA" => 6,
23         _ => panic!("unknown plant: {}", t),
24     }
25 }
26
27 pub fn water_boost(t: &str) -> i32 {
28     match t {
29         "PLUM" => 5,
30         "LEMON" => 5,
31         "APPLE" => 7,
32         "BANANA" => 2,
33         _ => panic!("unknown plant for water_boost: {}", t),
34     }
35 }
36
37 // — data structures —
38
39 pub type Cell = (i32, i32);
40
41 #[derive(Clone)]
42 pub struct Troll {
43     pub id: i32,
44     pub x: i32,
45     pub y: i32,
46     pub movement_speed: i32,
47     pub carry_capacity: i32,
48     pub harvest_power: i32,
49     pub chop_power: i32,
50     pub carry: [i32; 6],
51 }
52
53 impl Troll {
54     pub fn pos(&self) -> Cell {
55         (self.x, self.y)
56     }
57     pub fn total_carried(&self) -> i32 {
58         self.carry.iter().sum()
59     }
60     pub fn free_capacity(&self) -> i32 {
61         self.carry_capacity - self.total_carried()
62     }
63     pub fn stats(&self) -> (i32, i32, i32, i32) {
64         (
65             self.movement_speed,
66             self.carry_capacity,
67             self.harvest_power,
68             self.chop_power,
69         )
70     }
71 }
72
73 #[derive(Clone)]
74 pub struct Tree {
75     pub tree_type: String, // "PLUM", "LEMON", "APPLE", "BANANA"
76     pub x: i32,
77     pub y: i32,
78     pub size: i32,
79     pub health: i32,
80     pub fruits: i32,
81     pub cooldown: i32,
82 }
```

```

83
84 impl Tree {
85     pub fn pos(&self) -> Cell {
86         (self.x, self.y)
87     }
88 }
89
90 pub struct State {
91     pub walkable: HashSet<Cell>,
92     pub my_shack: Cell,
93     pub opp_shack: Cell,
94     pub my_inventory: [i32; 6],
95     pub opp_inventory: [i32; 6],
96     pub trees: Vec<Tree>,
97     pub my_trolls: Vec<Troll>,
98     pub opp_trolls: Vec<Troll>,
99     pub turn: i32,
100     pub iron_cells: HashSet<Cell>,
101     pub water_cells: HashSet<Cell>,
102 }
103
104 // — geometry helpers —————
105
106 pub const NEIGHBORS: [(i32, i32); 4] = [(0, 1), (1, 0), (0, -1), (-1, 0)];
107
108 pub fn ortho_neighbors(cell: Cell) -> [Cell; 4] {
109     let (x, y) = cell;
110     [(x, y + 1), (x + 1, y), (x, y - 1), (x - 1, y)]
111 }
112
113 pub fn is_adjacent(a: Cell, b: Cell) -> bool {
114     (a.0 - b.0).abs() + (a.1 - b.1).abs() == 1
115 }
116
117 // — BFS —————
118
119 pub fn bfs_distances(walkable: &HashSet<Cell>, sources: &[Cell]) -> HashMap<Cell, i32> {
120     let mut dist: HashMap<Cell, i32> = HashMap::new();
121     let mut queue: VecDeque<Cell> = VecDeque::new();
122     for &cell in sources {
123         if !dist.contains_key(&cell) {
124             dist.insert(cell, 0);
125             queue.push_back(cell);
126         }
127     }
128     while let Some((x, y)) = queue.pop_front() {
129         let d = dist[&(x, y)];
130         for &(dx, dy) in &NEIGHBORS {
131             let n = (x + dx, y + dy);
132             if walkable.contains(&n) && !dist.contains_key(&n) {
133                 dist.insert(n, d + 1);
134                 queue.push_back(n);
135             }
136         }
137     }
138     dist
139 }
140
141 pub fn manhattan(a: Cell, b: Cell) -> i32 {
142     (a.0 - b.0).abs() + (a.1 - b.1).abs()
143 }
144
145 // — training cost —————
146
147 pub fn training_cost(n: i32, talents: (i32, i32, i32, i32)) -> [i32; 6] {
148     let (ms, cc, hp, chop) = talents;
149     let mut cost = [0i32; 6];
150     cost[PLUM] = n + ms * ms;
151     cost[LEMON] = n + cc * cc;
152     cost[APPLE] = n + hp * hp;
153     cost[IRON] = n + chop * chop;
154     cost
155 }
156
157 pub fn afford_fruit_only(inv: &[i32; 6], cost: &[i32; 6]) -> bool {
158     inv[PLUM] >= cost[PLUM] && inv[LEMON] >= cost[LEMON] && inv[APPLE] >= cost[APPLE]
159 }
160
161 pub fn mb_afford(inv: &[i32; 6], cost: &[i32; 6], have_iron: bool) -> bool {
162     let iron_ok = !have_iron || inv[IRON] >= cost[IRON];
163     inv[PLUM] >= cost[PLUM] && inv[LEMON] >= cost[LEMON] && inv[APPLE] >= cost[APPLE] && iron_ok
164 }
165
166 pub fn ge_fruit_ty(t: &str) -> Option<usize> {
167     match t {
168         "PLUM" => Some(0),
169         "LEMON" => Some(1),
170         "APPLE" => Some(2),

```

```

171     "BANANA" => Some(3),
172     _ => None,
173 }
174 }
175
176 // — R5.0: deterministic tie-break "spread" —————
177 // v1.20.0's HashSet iteration accidentally gave RANDOM-PER-GAME tie-breaking, which
178 // spread plant spots around the shack; the R1 lexicographic determinization clustered
179 // them (measured ~0.5 arena vs same-hour baseline). These helpers restore the spread
180 // deterministically: a per-game salt from immutable map facts (STABLE WITHIN a game so
181 // tied targets never flap turn-to-turn) mixed with the cell -> a pseudo-random rank.
182
183 /// per-game-stable salt from immutable map facts (varies across maps/seats).
184 pub fn tie_salt(state: &State) -> u64 {
185     let (sx, sy) = state.my_shack;
186     let (ox, oy) = state.opp_shack;
187     let mut h = 0x9E37_79B9_7F4A_7C15u64;
188     for v in [
189         sx as u64,
190         sy as u64,
191         ox as u64,
192         oy as u64,
193         state.walkable.len() as u64,
194         state.water_cells.len() as u64,
195         state.iron_cells.len() as u64,
196     ] {
197         h ^= v.wrapping_mul(0x0000_0100_0000_01B3);
198         h = h.rotate_left(23).wrapping_mul(0xFF51_AFD7_ED55_8CCD);
199     }
200     h
201 }
202
203 /// mix a cell with the salt -> deterministic pseudo-random tie-break rank.
204 pub fn tie_mix(c: Cell, salt: u64) -> u64 {
205     let mut h = salt
206         ^ (c.0 as u64).wrapping_mul(0x9E37_79B9_7F4A_7C15)
207         ^ (c.1 as u64).wrapping_mul(0xC2B2_AE3D_27D4_EB4F);
208     h ^= h >> 33;
209     h = h.wrapping_mul(0xFF51_AFD7_ED55_8CCD);
210     h ^= h >> 29;
211     h
212 }

```

3 botmain/tactics.rs

L1 tactics — builds the per-turn Plan: phase selection, farm geometry (farm_d, front-door), constants.
(461 lines)

```
1  /// Tactics layer (L1, R4): everything decided BEFORE any troll is looked at – the
2  /// turn-1 adaptive chopper spec, train gating, farm geometry/phase, and the seed
3  /// reserve. `Plan` is the explicit L1-L2 interface consumed by jobs::assign_all.
4  /// Bodies moved VERBATIM from decide_elite; equality enforced by the harness.
5  use super::*;
6  use std::cell::RefCell;
7  use std::collections::{HashMap, HashSet};
8
9  #[derive(Clone, Copy, PartialEq, Eq, Debug)]
10 pub enum Meta {
11     Tempo,
12     Scale,
13 }
14
15 #[derive(Clone, Copy, PartialEq, Eq, Debug)]
16 pub enum Phase {
17     Tempo,
18     Hoard,
19     Factory,
20 }
21
22 /// Scale meta: hoard (no felling, bank the wallet) until T_SWITCH, then the factory.
23 /// Swept down per gatekeeper verdict #4 – hoarding until t=140 cedes the shared map to
24 /// deforesting opponents (opp wood 60.1 vs our 23.2 avg, boss avg score delta -169.0): 100 =
25 /// the design spec's sweep floor (docs/superpowers/specs/2026-07-07-last-mile-and-basin-jump-
26 /// design.md, "Risks": T_SWITCH sweeps down to 120/100 if Hoard loses the early race
27 /// unrecoverably). Note: SCALE_MIN_TURN stays [10, 40, 110] unchanged – slot 2 (the ladder's
28 /// only chop-capable hand) now has its min-turn gate (110) ABOVE T_SWITCH (100), so it becomes
29 /// eligible only after Factory has already begun. This is intentional: `ladder_funding`
30 /// (planner.rs; renamed from `scale_funding` in v1.35.0-thand) keeps the funding bands elevated
31 /// through a grace window scoped to `want_feeder`, not `phase == Phase::Hoard` alone, so the
32 /// ladder's tail still gets funded priority even though it crosses the Hoard->Factory boundary –
33 /// and, as of v1.35.0, the same grace window now also serves Tempo's own pending 3rd hand.
34 pub const T_SWITCH: i32 = 100;
35
36 pub fn phase_for(meta: Meta, turn: i32) -> Phase {
37     match meta {
38         Meta::Tempo => Phase::Tempo,
39         Meta::Scale => {
40             if turn < T_SWITCH {
41                 Phase::Hoard
42             } else {
43                 Phase::Factory
44             }
45         }
46     }
47 }
48
49 thread_local! {
50     /// v1.7.0: the chopper spec chosen ONCE at turn 1 from the starting draw.
51     static GE_CHOSEN_SPEC: RefCell<Option<(i32, i32, i32, i32)>> = RefCell::new(None);
52 }
53
54 /// Turn-1 reset of the committed spec.
55 pub fn reset() {
56     GE_CHOSEN_SPEC.with(|c| *c.borrow_mut() = None);
57 }
58
59 pub struct Plan {
60     pub shack: Cell,
61     /// MAP distances from the shack (BFS) – farm membership and chopper roam use THIS,
62     /// not manhattan (user-found bug vs biz1: manhattan ignores water, so the starter
63     /// planted "nearby" cells that were a long walk around a lake).
64     pub farm_d: std::collections::HashMap<Cell, i32>,
65     pub opp: Cell,
66     pub have_iron: bool,
67     pub turns_rem: i32,
68     pub n: i32,
69     pub farm_now: usize,
70     pub nchop: i32,
71     pub spec: (i32, i32, i32, i32),
72     pub want_chopper: bool,
73     pub want_feeder: bool,
74     pub train_spec: (i32, i32, i32, i32),
75     pub cost: [i32; 6],
76     pub train_now: bool,
77     pub need_iron: bool,
78     pub need_fund: [bool; 3],
79     pub farm_r: i32,
80     pub farm_cap: usize,
81     pub fell_size: i32,
82     pub farm_fell: i32,
```

```

83 pub chop_r: i32,
84 pub starter_chop: bool,
85 pub liquidation: bool,
86 pub base_trees: usize,
87 pub seed_cells: HashSet<Cell>,
88 pub phase: Phase,
89 /// v1.53.0-pressurefarm (Task 1 Step 3): the live ownership-pressure verdict, computed
90 /// ONCE per turn below (never recomputed in planner.rs's per-troll hot loop). Under
91 /// `PressureState::Green` this is always the all-zero/empty default and every
92 /// pressure-gated behavior in planner.rs is a proven no-op (see
93 /// tests/pressurefarm.rs::pressure_green_is_noop).
94 pub pressure: ownership::Pressure,
95 /// v1.54.0-frontdoor: the chosen "front door" cell when the shack straddles a
96 /// detected chokepoint (see `compute_door`); `None` on every normal map – a proven
97 /// no-op (tests/frontdoor.rs::frontdoor_open_map_noop).
98 pub door: Option<Cell>,
99 /// BFS distances from `door` ('Some' iff `door` is 'Some'). `farm_eligible` uses
100 /// THIS instead of `farm_d` at the farm/plant-membership sites exactly when a
101 /// chokepoint override is active; every OTHER use of `farm_d` (banking-adjacency,
102 /// chop-roam) is untouched.
103 pub door_d: Option<HashMap<Cell, i32>>,
104 }
105
106 // — FRONT-DOOR FARM PLACEMENT (v1.54.0-frontdoor) —————
107 // `farm_d` above is a BFS SEEDED AT THE SHACK CELL: the shack is impassable to trolls (they
108 // can never stand on / re-enter it), but the BFS still treats it as a zero-cost hub, so
109 // cells on OPPOSITE sides of a shack that straddles a chokepoint (lake + boulders, e.g. the
110 // Sasso map) both read farm_d<=2 even when they are 20+ REAL walking steps apart – one
111 // connected component, a DISTANCE bug, not a connectivity one (user replay 895493013,
112 // confirmed: farm_d(14,4)=1 but real dist (12,4)->(14,4)=24). The farm/plant-membership
113 // filter then wrongly admits cells on BOTH sides, so the gatherer shuttles the full detour
114 // every trip (263/300 turns in transit, measured).
115 //
116 // Fix: `compute_door` detects the straddle (chokepoint-gated) and, ONLY then, picks a
117 // single "front door" – the shack's walkable neighbor farthest (true BFS distance) from
118 // the OPPONENT shack among candidates VIABLE enough to host a farm. `farm_eligible` then
119 // resolves farm/plant membership through the door's BFS instead of the shack-hub `farm_d`.
120 // On every normal (non-chokepoint) map `compute_door` returns `(None, None)` and
121 // `farm_eligible` reduces to exactly today's `farm_d.get(pos) <= r` – a proven no-op (see
122 // tests/frontdoor.rs::frontdoor_open_map_noop). `farm_d` keeps ALL its other uses
123 // (banking-adjacency `farm_d==1` in planner.rs's plant_cell chooser, chop-roam `chop_r`)
124 // untouched – only the farm/plant-membership sites route through `farm_eligible`.
125 pub const MIN_FARM_CELLS: usize = 4; // a candidate door must host at least this many walkable cells within GE_FARM_R to be VIABLE
126 pub const CHOKE_THRESHOLD: i32 = 8; // max true pairwise distance between door candidates before we call it a chokepoint (open maps: ~4 via
127 // ↪ the small detour around the shack; Sasso: 24)
128
129 /// Chokepoint-gated front-door selection. Returns `(None, None)` on every normal map:
130 /// fewer than 2 walkable shack-neighbors (nothing to straddle), every candidate mutually
131 /// close (`<= CHOKE_THRESHOLD`), or no candidate is VIABLE (`>= MIN_FARM_CELLS` walkable
132 /// cells within `GE_FARM_R`). Otherwise picks the viable candidate maximizing true BFS
133 /// distance from `state.opp_shack` (farthest-from-enemy – the enemy must travel farther to
134 /// raid our crops), tie-broken lexicographically on the cell.
135 ///
136 /// Determinism: `candidates` is an explicit sorted `Vec` (never a HashSet iterated for
137 /// order); the viable list is explicitly sorted on `(-opp_dist, door_cell)` before picking
138 /// index 0 – the result depends only on map geometry, never on HashSet/HashMap internal
139 /// iteration order (see tests/frontdoor.rs::frontdoor_determinism_hashset_reorder).
140 pub fn compute_door(state: &State) -> (Option<Cell>, Option<HashMap<Cell, i32>>) {
141     let shack = state.my_shack;
142     let mut candidates: Vec<Cell> = ortho_neighbors(shack)
143         .into_iter()
144         .filter(|c| state.walkable.contains(c))
145         .collect();
146     candidates.sort();
147     if candidates.len() < 2 {
148         return (None, None); // nothing to straddle
149     }
150     let dds: Vec<(Cell, HashMap<Cell, i32>>) = candidates
151         .iter()
152         .map(|&c| (c, bfs_distances(&state.walkable, &[c])))
153         .collect();
154     let mut max_pair = 0;
155     for i in 0..dds.len() {
156         for j in (i + 1)..dds.len() {
157             let dist = dds[i].1.get(&dds[j].0).copied().unwrap_or(i32::MAX / 2);
158             max_pair = max_pair.max(dist);
159         }
160     }
161     if max_pair <= CHOKE_THRESHOLD {
162         return (None, None); // open map: no-op
163     }
164     let opp_d = bfs_distances(&state.walkable, &[state.opp_shack]);
165     let mut viable: Vec<(Cell, i32)> = Vec::new();
166     for (door, dd) in &dds {
167         let count = state

```

```

170         .walkable
171         .iter()
172         .filter(|c| dd.get(c).map_or(false, |&d| d <= GE_FARM_R))
173         .count();
174     if count >= MIN_FARM_CELLS {
175         viable.push((*door, opp_d.get(door).copied().unwrap_or(0)));
176     }
177 }
178 if viable.is_empty() {
179     return (None, None); // no side can host a farm: fall back to plain farm_d
180 }
181 viable.sort_by_key(|&(door, od)| (-od, door));
182 let chosen = viable[0].0;
183 let chosen_d = dds.into_iter().find(|(c, _)| *c == chosen).map(|(_, d)| d);
184 (Some(chosen), chosen_d)
185 }
186
187 // Farm/plant-cell eligibility at radius `r`: `farm_d <= r` on every normal map (byte-
188 // identical to the pre-frontdoor test); `door_d <= r` when a chokepoint override is
189 // active. The only call sites this replaces are farm/plant-membership tests (tactics.rs
190 // farm_now/base_trees below, planner.rs's plant_cell chooser) – NOT `farm_d==1` banking-
191 // adjacency or `chop_r` roam, which keep consulting `farm_d` directly.
192 pub fn farm_eligible(
193     farm_d: &HashMap<Cell, i32>,
194     door_d: &Option<HashMap<Cell, i32>>,
195     pos: Cell,
196     r: i32,
197 ) -> bool {
198     match door_d {
199         Some(dd) => dd.get(&pos).map_or(false, |&d| d <= r),
200         None => farm_d.get(&pos).map_or(false, |&d| d <= r),
201     }
202 }
203
204 fn plan_impl(state: &State, my: &[Troll], meta: Meta) -> Plan {
205     let farm_d = bfs_distances(&state.walkable, &[state.my_shack]);
206     let (door, door_d) = compute_door(state);
207     let shack = state.my_shack;
208     let opp = state.opp_shack;
209     let inv = &state.my_inventory;
210     let have_iron = !state.iron_cells.is_empty();
211     let turns_rem = TOTAL_TURNS - state.turn + 1;
212
213     let n = my.len() as i32;
214
215     // — training: ONE chopper, EARLY, spec ADAPTED to the starting draw (v1.7.0) —
216     // Denial > production (proven 2026-07-05): the chopper must train at turn 1 so we win
217     // the shared-tree race. cc=3 captures 3 wood/tree but its lemon cost (n+9=10) only fits
218     // a rich starting draw; cc=2 (lemon n+4=5) trains at t1 on nearly any draw. So pick the
219     // RICHEST chopper the turn-1 inventory can train IMMEDIATELY – cc=3 when the draw affords
220     // it (production), else cc=2 (denial) – NEVER delay waiting to afford cc=3.
221     let spec = GE_CHOSEN_SPEC.with(|c| {
222         let mut c = c.borrow_mut();
223         if c.is_none() {
224             // v1.9.0 (data-driven vs 32 real Boss-5 games): pick EACH axis independently to
225             // level 3 iff the turn-1 draw's binding resource already affords it – ms<-plum,
226             // cc<-lemon, chop<-iron (each level-3 costs n+9). This (a) FIXES the v1.7.0 bug
227             // where a plum/iron shortfall wrongly locked cc2 on a lemon-rich map (cc3 vs cc2
228             // differ ONLY in lemon), and (b) adopts Boss 5's ms=3/chop=3 flexibility (faster
229             // travel + faster felling = the sustained-throughput lever it beats us on). An axis
230             // upgrades only when its resource is ALREADY >= n+9, so it never delays training
231             // beyond the cc2 baseline (the upgrade is "free"). hp=0 (can't harvest, cheap).
232             // ms/cc/chop upgrade to 3 when their resource is free (>=n+9), else 2. (v1.14.0's ccl
233             // tier on lemon-poor maps was WORSE: 0/5 wood 40 – ccl throughput too low even with
234             // the tight farm's cheap banking. A late cc2 beats an early ccl.)
235             let lvl = |res: usize| if inv[res] >= n + 9 { 3 } else { 2 };
236             *c = Some((lvl(PLUM), lvl(LEMON), 0, lvl(IRON)));
237         }
238         c.unwrap()
239     });
240     let farm_now = state
241         .trees
242         .iter()
243         .filter(|p| farm_eligible(&farm_d, &door_d, p.pos(), GE_FARM_R))
244         .count();
245     // v1.11.0: troll 2 = the CHOPPER (early, adaptive spec). troll 3 = a FEEDER (late): a cheap
246     // hp>0/chop=0 harvester. Because decide_elite routes any chop<2 troll through the STARTER
247     // (printer) branch, the feeder auto-plants bananas – a 2nd pair of hands keeping the farm
248     // DENSE so the single chopper never travels/idles (travel is ~2.5x the felling = the real
249     // Boss-5 throughput gap). This is the runninglvlan structure (starter+feeder+chopper) and
250     // AVOIDS the 2-chopper starvation (validated: a 2nd chopper starves the 1-feeder farm).
251     let chop = my.iter().filter(|u| u.chop_power >= 2).count() as i32;
252     // B2 (Scale ladder): under Meta::Scale, replace the adaptive chopper-training logic with a
253     // FIXED HAND ladder – want_chopper forced false (the early adaptive t1 chopper is REPLACED
254     // by the ladder itself: its final slot trains a real chopper `(2,2,0,2)` once n reaches 3
255     // hands, gated at t>=110; Hoard banks the wallet with hands only before that). Troll count n
256     // selects the next hand's spec/turn-gate from
257     // SCALE_LADDER/SCALE_MIN_TURN, mapped onto the SAME Plan fields the Tempo path uses

```

```

258 // (want_feeder/train_spec/cost/train_now/need_iron/need_fund) so planner.rs needs no new
259 // fields – it already reads want_feeder to drive funding/printer work. The Tempo branch
260 // below is BYTE-IDENTICAL to the pre-B2 code (equality-critical: GE_META stays Tempo live).
261 let (want_chopper, want_feeder, train_spec, cost, train_now, need_iron, need_fund) = if meta
262 == Meta::Scale
263 {
264     const SCALE_LADDER: [(i32, i32, i32, i32); 3] = [(1, 1, 1, 0), (1, 1, 1, 0), (2, 2, 0, 2)];
265     const SCALE_MIN_TURN: [(i32; 3)] = [10, 40, 110];
266     let slot = ((n - 1).max(0) as usize).min(2);
267     let want_hand = n < 4 && state.turn >= SCALE_MIN_TURN[slot];
268     let want_chopper = false;
269     let want_feeder = want_hand;
270     let train_spec = SCALE_LADDER[slot];
271     let cost = training_cost(n, train_spec);
272     let train_now = want_hand && mb_afford(inv, &cost, have_iron);
273     // B2.1 gatekeeper fix: accumulate iron EARLY (all of Hoard), not only once slot 2 is
274     // reached at t>=110 – 7 = the slot-2 chopper's iron cost (n=3 + chop^2=4). Iron income
275     // is otherwise zero (nothing mines it) and the map's starting stock rarely reaches 7 by
276     // t110, so the wallet must be pre-filled during Hoard or the ladder's only chop-capable
277     // hand never trains (wood=0 the entire game, confirmed 12/12 in the gatekeeper report).
278     let need_iron = have_iron && inv[IRON] < 7;
279     let need_fund: [bool; 3] = [inv[0] < cost[0], inv[1] < cost[1], inv[2] < cost[2]];
280     (
281         want_chopper,
282         want_feeder,
283         train_spec,
284         cost,
285         train_now,
286         need_iron,
287         need_fund,
288     )
289 } else {
290     let want_chopper = nchop == 0 && (state.turn >= GE_CHOP_DELAY || farm_now >= GE_CHOP_FARM);
291     let want_feeder = nchop >= 1
292         && n < GE_MAX_TROLLS
293         && state.turn >= GE_FEEDER_T
294         && farm_now >= GE_FEEDER_FARM;
295     let train_spec = if want_chopper { spec } else { GE_FEEDER_SPEC };
296     let cost = training_cost(n, train_spec);
297     let train_now = (want_chopper || want_feeder) && mb_afford(inv, &cost, have_iron);
298     let want_chopper = want_chopper; // (kept: need_iron/need_fund below key off the pending hand)
299     // iron-gated: fruit is ready but we still lack the iron for the PENDING HAND – the
300     // chopper OR the feeder. T-hand.1 (gatekeeper v1.35.0 verdict, fix a): this used to be
301     // want_chopper-only, so iron mining stopped FOREVER the instant the chopper trained,
302     // permanently starving any later pending hand of its flat cost[IRON]=n training cost
303     // (every spec carries it) on every iron-bearing map – 12/12 sampled by the gatekeeper.
304     let need_iron = have_iron
305         && (want_chopper || want_feeder)
306         && inv[IRON] < cost[IRON]
307         && afford_fruit_only(inv, &cost);
308     // which fruit types still block the pending hand (funding targets)
309     let need_fund: [bool; 3] = [inv[0] < cost[0], inv[1] < cost[1], inv[2] < cost[2]];
310     (
311         want_chopper,
312         want_feeder,
313         train_spec,
314         cost,
315         train_now,
316         need_iron,
317         need_fund,
318     )
319 };
320
321 // — farm config —————
322 // v1.18.0: TURN-1 ADAPTIVE ECONOMY (committed via the draw-chosen spec, no mid-game switch).
323 // High-draw map (spec chose cc>=3): economy B = BIG farm + size-3 fells (cc3 captures 3, chop3
324 // fells size-3 in 2 chops; the cc3 banks-every-3 offsets the bigger farm's longer trips) – the
325 // Boss-5 throughput economy. Low-draw map (cc2): economy A = the TIGHT farm (short bank trips,
326 // fast size-2 maturation) that beats Boss 5 ~40%. Best of both, per the felling mechanics.
327 let phase = phase_for(meta, state.turn);
328 let econ_b = false; // econ B (big-farm size-3) arena-validated WORSE (135 vs 120) – the big farm cannot sustain size-3 maturation;
329 // pure tight-farm (A) is best
330 let farm_r = if econ_b { 3 } else { GE_FARM_R };
331 // B3 (Factory): once the Scale meta reaches Factory (post-T_SWITCH), the hoard-built wallet
332 // funds a bigger farm – 20 slots instead of 12 – so the plant-and-fell loop has room to grow
333 // with the trained hand ladder. Hoard/Tempo are unchanged (econ_b is a permanent `false`, so
334 // they fall through to GE_FARM_MAX=12).
335 let farm_cap = if phase == Phase::Factory {
336     20
337 } else if econ_b {
338     20
339 } else {
340     GE_FARM_MAX
341 };
342 let fell_size = GE_FELL_SIZE; // NATIVE/contested trees: always size-2 = DENIAL
343 let farm_fell = if econ_b { 3 } else { 2 }; // OUR farm bananas: size-3 in econ B, size-2 in A
344 let chop_r = if econ_b { 10 } else { GE_CHOP_R }; // econ B roams a bigger farm; A stays tight
345 let starter_chop = GE_STARTER_CHOP;

```

```

345 let liquidation = turns_rem <= GE_LIQ_T;
346 let base_trees = state
347   .trees
348   .iter()
349   .filter(|p| farm_eligible(&farm_d, &door_d, p.pos(), farm_r))
350   .count();
351
352 // — SEED SUSTAINABILITY (arena deforestation fix) —————
353 // Trees only fruit at MAX_SIZE(4); felling farm bananas at size 2 means they
354 // NEVER fruit, so the seed supply drains -> the farm dies -> our half
355 // deforests -> both trolls park (the decoded arena stall). Fix: keep the K
356 // most-mature farm bananas as a permanent seed reserve the chopper won't
357 // fell — they ripen, fruit, and the starter harvests their fruit for seeds.
358 let mut seed_cells: HashSet<Cell> = HashSet::new();
359 if GE_SEED_RESERVE > 0 && !liquidation {
360   let mut fb: Vec<&Tree> = state
361     .trees
362     .iter()
363     .filter(|p| {
364       p.tree_type == "BANANA" && farm_d.get(&p.pos()).map_or(false, |&d| d <= farm_r)
365     })
366     .collect();
367   fb.sort_by_key(|p| (-p.size, -p.fruits, manhattan(p.pos(), shack), p.pos()));
368   for p in fb.into_iter().take(GE_SEED_RESERVE) {
369     seed_cells.insert(p.pos());
370   }
371 }
372
373 let provisional = Plan {
374   shack,
375   farm_d,
376   opp,
377   have_iron,
378   turns_rem,
379   n,
380   farm_now,
381   nchop,
382   spec,
383   want_chopper,
384   want_feeder,
385   train_spec,
386   cost,
387   train_now,
388   need_iron,
389   need_fund,
390   farm_r,
391   farm_cap,
392   fell_size,
393   farm_fell,
394   chop_r,
395   starter_chop,
396   liquidation,
397   base_trees,
398   seed_cells,
399   phase,
400   pressure: ownership::Pressure::default(),
401   door,
402   door_d,
403 };
404
405 // — PRESSURE GOVERNOR (v1.53.0-pressurefarm, Task 1 Step 2) —————
406 // ownership::assess only reads provisional.farm_d/farm_r/seed_cells (all already final
407 // above) — the placeholder `pressure` field on `provisional` is never read by it, so
408 // computing against the provisional Plan and overlaying the real pressure (plus its one
409 // derived override, farm_cap) afterward is equality-safe. Computed exactly ONCE per
410 // turn here — never inside planner.rs's per-troll candidates() hot loop.
411 let pressure = ownership::assess(state, &provisional);
412 // Task 2 Step 1 (dynamic farm cap): Orange+ pressure suppresses further expansion, but
413 // NEVER below a small survival floor — a farm already at/under the floor keeps planting
414 // regardless (the `.min` only ever shrinks the CEILING, it can't force liquidation). This
415 // keeps Green/Yellow byte-identical (provisional.farm_cap is returned unchanged) and
416 // avoids the "always smaller farm" static-control trap: the clamp only engages when
417 // pressure is actually observed.
418 //
419 // Code review C2 (2026-07-09): re-gated from `>= Yellow` to `>= Orange`. Yellow only
420 // requires `own_half_exposed > 0` (created_exposed == 0) — a signal that lights up from
421 // static map geometry (any own-half tree we can't PROVE decisively ours) and is
422 // near-permanent from ~turn 5 on real maps, independent of any real threat to farm value
423 // WE created. Gating the clamp there collapsed farm_cap 12->4 for essentially the whole
424 // game — exactly the "always smaller farm" nerf the paragraph above warns against, and a
425 // throughput crater (dense-farm-never-idle is this bot's whole economic thesis). Orange
426 // requires `created_exposed > 0` — a created/local farm tree the ownership model itself
427 // marks not-safely-ours — which IS threat-discriminating (it needs the opponent's ETA to
428 // actually contest a tree WE planted), matching this feature's own design intent (see
429 // docs/pressure-aware-farm.md Task 0 Step 3, "Yellow: ... pause expansion ONLY IF
430 // created/local value exists") and data/analysis/map-value-ownership/report.md's
431 // recommended trigger.
432 //

```



```

433 // Factory latent note (M1): under Phase::Factory the champion raises farm_cap to 20
434 // (see the `phase == Phase::Factory` branch above); this clamp would override that down
435 // to GE_PRESSURE_FARM_FLOOR if Orange+ pressure ever fires during Factory. Dormant today
436 // (GE_META=Tempo, Factory unreachable) – flagged, not handled; no logic added here.
437 let farm_cap = if pressure.state >= ownership::PressureState::Orange {
438     provisional.farm_cap.min(GE_PRESSURE_FARM_FLOOR)
439 } else {
440     provisional.farm_cap
441 };
442
443 Plan {
444     farm_cap,
445     pressure,
446     ..provisional
447 }
448 }
449
450 pub fn plan(state: &State, my: &[Troll]) -> Plan {
451     plan_impl(state, my, super::GE_META)
452 }
453
454 /// Test-only seam: drive `plan_impl` under an explicit `Meta` instead of the compile-time
455 /// `GE_META` const. Plain `pub` (not `cfg(test)`) because integration tests in `rust/tests/`
456 /// compile as a separate crate and can't see `cfg(test)` items; this is dead code in the arena
457 /// build, which the crate's `#![allow(dead_code, unused)]` already tolerates, and the bundler
458 /// carries it harmlessly (submission size gate is on the minified bytes, not source LOC).
459 pub fn plan_with_meta(state: &State, my: &[Troll], meta: Meta) -> Plan {
460     plan_impl(state, my, meta)
461 }

```

4 botmain/planner.rs

L2 planner — the joint TASK manager: candidates() produces per-troll value-banded tasks, assign_resolved() matches them conflict-free, race/sticky/yield refinements.

(1156 lines)

```
1  /// L2 JOINT TASK ASSIGNMENT (R6b) — the activity manager's task stage.
2  ///
3  /// The sequential cascade (jobs.rs) let troll-id order decide contested resources: the
4  /// first troll `reserved` its target, later trolls avoided it. Here every troll's viable
5  /// tasks are enumerated as (target, value) CANDIDATES — the cascade's branch hierarchy
6  /// becomes value BANDS (spaced wider than any ETA, so each troll still prefers its higher
7  /// branch), ETA differentiates within a band — and the assignment is chosen JOINTLY:
8  /// exhaustive over per-troll top-K, maximizing total value, conflicting target claims
9  /// forbidden, canonical tie-break. SHUFFLE INVARIANCE: the plan depends on the objective,
10 /// never on troll/candidate iteration order.
11 use super::tactics::{farm_eligible, Phase, Plan};
12 use super::*;
13 use std::cell::RefCell;
14 use std::collections::{HashMap, HashSet};
15
16 thread_local! {
17     /// last MoveTo target per troll (diagnostics: assignment-flap counter) + flap count
18     static LAST_TGT: RefCell<HashMap<i32, Cell>> = RefCell::new(HashMap::new());
19     static FLAPS: RefCell<u32> = RefCell::new(0);
20 }
21
22 /// Turn-1 reset of diagnostics.
23 pub fn reset() {
24     LAST_TGT.with(|m| m.borrow_mut().clear());
25     FLAPS.with(|f| *f.borrow_mut() = 0);
26 }
27
28 pub fn flaps() -> u32 {
29     FLAPS.with(|f| *f.borrow())
30 }
31
32 const K: usize = 8; /// per-troll candidate cap (bands make more irrelevant)
33 const BAND: i64 = 100_000; /// > any ETA by orders of magnitude
34     /// v1.28.1 STICKINESS: bonus for keeping last turn's target — the joint matcher re-plans
35     /// globally every turn and small ETA shifts flipped assignments mid-travel (measured 16-36
36     /// flaps/game = leaked steps, the v1.27 arena fade). Within-band (« BAND): stability never
37     /// overrides the priority hierarchy, only breaks near-ties toward the current plan.
38 const STICKY: i64 = 6; /// v1.28.3 sweep: residual flaps 2-21 at 3; absorb bigger ETA jitter
39     /// v1.38.0-deny1 (A2 probe): bias the PRIMARY fell choice (bands 70/72 ONLY — not the
40     /// anti-starvation fallback, not the starter's chop-help band) toward trees nearer the
41     /// opponent's shack. Silver-era denial weighting toward the foe was the single biggest lever
42     /// measured pre-planner (MB_DENIAL_W in botmain.rs); the R6b joint planner has carried weight
43     /// 0 since it replaced that cascade. At DENY_W=0 every fell value is byte-identical to the
44     /// pre-probe code (the subtracted term is `0 * x == 0`); DENY_W=1 only breaks near-ties and
45     /// nudges marginal calls, « BAND — never overrides the priority hierarchy.
46     /// v1.39.0-sharepen4: REVERTED — analyst b62c977 measured this candidate at ~17.0 (down from
47     /// the 19.9-20.1 race-check band) and diagnosed a collision with the race check's own
48     /// tie-breaking. Parked at 0 (byte-identical to pre-probe) pending a retest that doesn't fight
49     /// RACE_SHARE_PEN; see tests/deny_probe.rs (its one test now requires DENY_W=1 and is ignored).
50 const DENY_W: i64 = 0; /// A2 reverted — collided with the race check per analyst b62c977; knob kept at 0
51     /// v1.36.0-race: mild discount for a JOINABLE contested tree (an enemy is already chopping it,
52     /// but we can arrive before they finish) — the wood splits round-robin among cell-sharers
53     /// (engine apply_chop), so a shared tree is worth slightly less than an uncontested one, but
54     /// never enough to lose to a materially worse alternative. « BAND, like STICKY.
55     /// v1.39.0-sharepen4: sweep 2 -> 4 per analyst (queue #1, b62c977) — the race check is the one
56     /// mechanism that just gained +1.3 in the arena; the analyst's decoded losses show excessive
57     /// trekking to contested trees when a free tree is only marginally farther away, so discount
58     /// joinable contests harder.
59 const RACE_SHARE_PEN: i64 = 2; /// sharepen4 kept-at-parity = INCONCLUSIVE under policy v2; champion (race) semantics = 2
60 /// v1.53.0-pressurefarm (Task 2 Step 3): under Orange/Red observed pressure, a created/farm
61 /// tree the ownership model marks not-safely-ours (plan.pressure.exposed_created_cells) gets
62 /// a small within-band bump — raises it before less-urgent same-band work, never overrides
63 /// the priority hierarchy (« BAND, same discipline as STICKY/DENY_W/RACE_SHARE_PEN above).
64 /// Under Green/Yellow, exposed_created_cells is always empty (Yellow's own_half signal alone
65 /// never implies created_exposed>0 — see ownership::classify_pressure), so this is always +0
66 /// there: a proven no-op, not a static preference.
67 const PRESSURE_LIQ_BONUS: i64 = 4;
68
69 #[derive(Clone, Debug, PartialEq)]
70 enum Kind {
71     Bank, /// render via motion::bank_cmd (DROP if adjacent, else camp-cell MOVE)
72     Park, /// render via motion::park_cmd (target None = idle band-10, ring-2-aware;
73     /// target Some(shack) = band-49 park-to-pick errand, direct camp approach)
74     ChopHere, /// CHOP at current cell
75     MoveTo, /// MOVE toward target (fell/fund/seed/mine-adjacent/plant travel)
76     PlanHere, /// PLANT BANANA at current cell
77     Harvest, /// HARVEST at current cell
78     Mine, /// MINE (adjacent to iron)
79     Pick, /// PICK BANANA (shack-adjacent)
80 }
81
```

```

82  #[derive(Clone, Debug)]
83  struct Cand {
84      kind: Kind,
85      target: Option<Cell>, // claimed resource (tree/plant/iron-adj cell); None = un-contested
86      value: i64,
87  }
88
89  #[derive(Clone, Copy, Debug, PartialEq, Eq)]
90  enum ClaimClass {
91      Cell,
92      Fruit,
93      Wood,
94  }
95
96  #[derive(Clone, Copy, Debug)]
97  struct ClaimInfo {
98      class: ClaimClass,
99      cell: Cell,
100     steps: i64,
101 }
102
103  #[derive(Clone, Debug)]
104  struct Assignments {
105     ids: Vec<i32>,
106     cands: Vec<Vec<Cand>>,
107     picks: Vec<usize>,
108 }
109
110  impl Assignments {
111     fn idx(&self, id: i32) -> Option<usize> {
112         self.ids.binary_search(&id).ok()
113     }
114
115     fn selected(&self, id: i32) -> Option<&Cand> {
116         let i = self.idx(id)?;
117         self.cands.get(i)?.get(self.picks[i])
118     }
119
120     fn selected_value(&self, id: i32) -> Option<i64> {
121         self.selected(id).map(|c| c.value)
122     }
123 }
124
125  fn eta(d: &HashMap<Cell, i32>, c: Cell, ms: i32) -> i64 {
126     let dist = d.get(&c).copied().unwrap_or(1 << 20);
127     ((dist + ms - 1) / ms.max(1)) as i64
128 }
129
130  fn value_band(value: i64) -> i64 {
131     (value + BAND - 1) / BAND
132 }
133
134  fn claim_info(state: &State, c: &Cand, steps: i64) -> Option<ClaimInfo> {
135     let cell = c.target?;
136     let class = match c.kind {
137         Kind::ChopHere => ClaimClass::Wood,
138         Kind::Harvest => ClaimClass::Fruit,
139         Kind::MoveTo => {
140             let targets_tree = state.trees.iter().any(|p| p.pos() == cell);
141             if targets_tree {
142                 match value_band(c.value) {
143                     70 | 40 | 30 => ClaimClass::Wood,
144                     63 | 62 | 58 | 52 | 44 | 38 => ClaimClass::Fruit,
145                     _ => ClaimClass::Cell,
146                 }
147             } else {
148                 ClaimClass::Cell
149             }
150         }
151         Kind::Bank | Kind::Park | Kind::PlantHere | Kind::Mine | Kind::Pick => ClaimClass::Cell,
152     };
153     Some(ClaimInfo { class, cell, steps })
154 }
155
156  fn claims_conflict(a: ClaimInfo, b: ClaimInfo) -> bool {
157     if a.cell != b.cell {
158         return false;
159     }
160     match (a.class, b.class) {
161         (ClaimClass::Fruit, ClaimClass::Wood) => a.steps >= b.steps,
162         (ClaimClass::Wood, ClaimClass::Fruit) => b.steps >= a.steps,
163         _ => true,
164     }
165 }
166
167  /// candidates for one troll – a faithful transcription of the jobs.rs cascade into bands.
168  #[allow(clippy::too_many_lines)]
169  fn candidates(state: &State, plan: &Plan, my: &[Troll], u: &Troll, salt: u64) -> Vec<Cand> {

```

```

170 let shack = plan.shack;
171 let inv = &state.my_inventory;
172 let d = bfs_distances(&state.walkable, &[u.pos()]);
173 let ms = u.movement_speed;
174 let is_chopper = u.chop_power >= 2;
175 let mut out: Vec<Cand> = Vec::new();
176
177 // B2 (Hoard): suppress felling except the denial emergency (an enemy troll within
178 // map-distance 2 of the tree). Computed ONCE per candidates() call – a single multi-source
179 // BFS from every opp troll, not a BFS per (enemy, tree) pair – and ONLY during Hoard, so
180 // the Tempo path (the live meta) pays zero extra cost.
181 let hoard = plan.phase == Phase::Hoard;
182 let enemy_d: Option<HashMap<Cell, i32>> = if hoard {
183     Some(bfs_distances(
184         &state.walkable,
185         &state.opp_trolls.iter().map(|e| e.pos()).collect::<Vec<_>>(),
186     ))
187 } else {
188     None
189 };
190 let threatened = |pc: Cell| -> bool {
191     enemy_d
192         .as_ref()
193         .map_or(false, |ed| ed.get(&pc).map_or(false, |&dd| dd <= 2))
194 };
195
196 let fell_ok = |p: &Tree| -> bool {
197     // v1.53.0-pressurefarm (Task 2 Step 2): a protected seed tree stays protected UNLESS
198     // the pressure governor has specifically released it (Orange/Red AND this exact
199     // tree is definitively not ours – see ownership::Pressure::released_seed_cells's doc
200     // comment for why the release check is per-tree, not the broader "exposed" set).
201     // Under Green/Yellow/Orange-without-a-definite-loser, released_seed_cells is always
202     // empty, so this is byte-identical to the pre-pressure check.
203     if plan.seed_cells.contains(&p.pos()) && !plan.pressure.released_seed_cells.contains(&p.pos())
204     {
205         return false;
206     }
207     if plan.liquidation {
208         return p.size >= 1;
209     }
210     let farm_banana = p.tree_type == "BANANA"
211         && plan
212             .farm_d
213             .get(&p.pos())
214             .map_or(false, |&fd| fd <= plan.farm_r);
215     p.size
216         >= if farm_banana {
217             plan.farm_fell
218         } else {
219             plan.fell_size
220         }
221 };
222 let own_half =
223     |p: &Tree| plan.liquidation || manhattan(p.pos(), shack) <= manhattan(p.pos(), plan.opp);
224 let within_roam = |p: &Tree| {
225     plan.liquidation
226     || plan
227         .farm_d
228         .get(&p.pos())
229         .map_or(false, |&fd| fd <= plan.chop_r)
230 };
231 // v1.36.0-race (user replay finding): a tree an enemy is already chopping is a RACE.
232 // If they fell it before we arrive, walking there donates the travel (skip). If we can
233 // arrive in time, the wood SPLITS round-robin among cell-sharers (engine apply_chop) –
234 // join, but discount the value by the shared payoff. Pure function of `state` (no
235 // per-troll mutable state), so shuffle invariance holds; called once per candidate,
236 // covers every fell-type push (bands 72/70, 42/40, 31/30) via this one helper.
237 let race = |pc: Cell, our_eta: i64| -> Option<i64> {
238     // returns None = doomed (skip candidate); Some(penalty) = value adjustment
239     let occupant = state
240         .opp_trolls
241         .iter()
242         .find(|e| e.pos() == pc && e.chop_power > 0);
243     match occupant {
244         None => Some(0),
245         Some(e) => {
246             let h = state
247                 .trees
248                 .iter()
249                 .find(|p| p.pos() == pc)
250                 .map(|p| p.health)
251                 .unwrap_or(0) as i64;
252             let their_turns = (h + e.chop_power as i64 - 1) / e.chop_power.max(1) as i64;
253             if their_turns <= our_eta {
254                 None // they finish first: doomed
255             } else {
256                 Some(RACE_SHARE_PEN) // joinable: shared wood, mild discount
257             }
258         }
259     }
260 }

```

```

258     }
259 }
260 };
261 // v1.53.0-pressurefarm (Task 2 Step 3): see PRESSURE_LIQ_BONUS's doc comment above.
262 //
263 // Code review I1 (2026-07-09): `race_pen` must gate this too. PRESSURE_LIQ_BONUS (4) >
264 // RACE_SHARE_PEN (2), so applying the bonus unconditionally on a joinable-contested tree
265 // (race_pen == RACE_SHARE_PEN) would more than cancel that discount (net +2), REVERSING
266 // the race check's tuned "don't over-trek to a shared/discounted tree" behavior into a
267 // preference for it – the exact opposite of what v1.36.0-race earned its +1.3. A doomed
268 // tree (race() returned None) never reaches here at all (every call site `continue`s on
269 // None before computing race_pen or calling this), so the only two live values of
270 // race_pen are 0 (no opponent occupant – genuinely non-contested) and RACE_SHARE_PEN (a
271 // joinable race). Withholding the bonus whenever race_pen != 0 therefore fully preserves
272 // it on every non-contested exposed tree (this behavior's primary job – raise exposed
273 // farm trees so we fell them before the opponent arrives) while making a contested tree's
274 // net adjustment exactly `-race_pen` either way, never a reversal.
275 let pressure_bonus = |pc: Cell, race_pen: i64| -> i64 {
276   if race_pen != 0 {
277     return 0;
278   }
279   if plan.pressure.state >= ownership::PressureState::Orange
280     && plan.pressure.exposed_created_cells.contains(&pc)
281   {
282     PRESSURE_LIQ_BONUS
283   } else {
284     0
285   }
286 };
287
288 // plant-cell search (shared across bands 80/88/50/49): the best free base cell within
289 // the farm radius, reachable from this troll. Computed ONCE, whenever there's farm room
290 // (base_trees < farm_cap), for EVERY troll – chopper included; band 80 just below needs
291 // the answer even though only a non-chopper carrying a banana can ever act on it – so
292 // band 80 (full -> bank), band 88 (plant the carried banana) and the PICK/park-to-pick
293 // bands (50/49, both in the STARTER branch further down) all agree on whether a banana
294 // would even be usable.
295 // v1.41.0-nopickloop (user-observed corridor livelock): on maps where water + the map
296 // edge leave no reachable, tree-free, un-occupied cell within the farm radius (a
297 // dead-end pocket, or a shack whose few walkable neighbors are all tree/troll-occupied),
298 // the OLD code still issued PICK whenever the tent held a banana. The picked banana then
299 // had nowhere to plant; band 80 (full->bank) was suppressed for a banana-carrying
300 // starter expecting to plant it (gated on the tree-COUNT `base_trees < farm_cap`, not a
301 // free-CELL check – the bug's heart), so the fallback band 10 banked it right back next
302 // turn, and PICK fired again the turn after – an infinite PICK<->DROP loop that also
303 // parked the starter on a scarce shack-adjacent cell the chopper needs for banking.
304 // Gating bands 88/50/49 on `plant_cell.is_some()` fixed the PICK half; band 80 below,
305 // gated the same way (reviewer MINOR fix), closes the other half – a carried banana with
306 // nowhere reachable to plant is banked, not hoarded waiting for a spot that never opens.
307 let plant_cell: Option<Cell> = if plan.base_trees < plan.farm_cap {
308   state
309     .walkable
310     .iter()
311     .filter(|c| {
312       farm_eligible(&plan.farm_d, &plan.door_d, **c, plan.farm_r) && d.contains_key(*c)
313     })
314     .filter(|c| !state.trees.iter().any(|p| p.pos() == **c))
315     .filter(|c| !my.iter().any(|o| o.id != u.id && o.pos() == **c))
316     .min_by_key(|c| {
317       let wet = state.water_cells.iter().any(|w| manhattan(*w, **c) == 1);
318       // v1.37.0-nanaflow (user replay finding #3): DIAGONAL PLANT PLACEMENT. The
319       // four cells orthogonally adjacent to the shack (farm_d==1) are the only
320       // bank/DROP cells every hand's carry trip needs – planting there congests
321       // banking, so penalize them (+3). The four diagonal-to-shack cells sit at
322       // the same map-distance (2) but off that traffic path, so reward them (-1).
323       let bank_adj = plan.farm_d.get(*c).copied() == Some(1);
324       let (cx, cy) = **c;
325       let diag = (cx - plan.shack.0).abs() == 1 && (cy - plan.shack.1).abs() == 1;
326       let geo = (if bank_adj { 3 } else { 0 }) + (if diag { -1 } else { 0 });
327       (d[*c] + if wet { 0 } else { 2 } + geo, tie_mix(**c, salt))
328     })
329     .copied()
330 } else {
331   None
332 };
333
334 // endgame banking (band 95): bank a carried load in time to score it
335 if u.total_carried() > 0 {
336   let d_home = ortho_neighbors(shack)
337     .iter()
338     .filter(|c| state.walkable.contains(*c))
339     .filter_map(|c| d.get(c))
340     .min()
341     .copied()
342     .unwrap_or(i32::MAX / 2);
343   let e = ((d_home + ms - 1) / ms.max(1) + 1) as i64;
344   if (plan.turns_rem as i64) <= e + 1 {
345     out.push(Cand {

```

```

346         kind: Kind::Bank,
347         target: None,
348         value: 95 * BAND - e,
349     });
350 }
351 }
352 // full -> bank (band 80) - reviewer MINOR fix: was `plan.base_trees < plan.farm_cap` (a
353 // tree COUNT), now `plant_cell.is_some()` (an actual reachable free CELL), matching the
354 // gate bands 88/50/49 already use. A carried banana with no plantable cell should be
355 // banked, not held waiting for room that will never materialize.
356 if u.free_capacity() == 0 && !(is_chopper && u.carry[BANANA] > 0 && plant_cell.is_some()) {
357     out.push(Cand {
358         kind: Kind::Bank,
359         target: None,
360         value: 80 * BAND,
361     });
362 }
363
364 if is_chopper {
365     // fell targets (band 70): standing (CHOP now) or travel; value differentiates by
366     // steps + chop-time exactly like the cascade's nearest_fell metric.
367     for p in state
368         .trees
369         .iter()
370         .filter(|p| fell_ok(p) && own_half(p) && within_road(p))
371     {
372         let pc = p.pos();
373         if !d.contains_key(&pc) {
374             continue;
375         }
376         if hoard && !threatened(pc) {
377             continue; // Hoard: no fells unless the tree is under denial threat
378         }
379         let steps = eta(&d, pc, ms);
380         let chop_t = ((p.health + u.chop_power.max(1) - 1) / u.chop_power.max(1)) as i64;
381         let race_pen = match race(pc, steps) {
382             None => continue, // doomed: they fell it before we arrive - skip, don't donate the travel
383             Some(pen) => pen,
384         };
385         // A2 probe (DENY_W): trees closer to the opponent lose less -> rank higher.
386         let deny_pen = DENY_W * (manhattan(pc, plan.opp) as i64 / 2);
387         let pbonus = pressure_bonus(pc, race_pen);
388         if pc == u.pos() {
389             // standing on a felleable tree: FINISH IT (cascade branch order) - band 72
390             // outranks every travel-fell so invested chops are never abandoned.
391             out.push(Cand {
392                 kind: Kind::ChopHere,
393                 target: Some(pc),
394                 value: 72 * BAND - chop_t - race_pen - deny_pen + pbonus,
395             });
396         } else {
397             out.push(Cand {
398                 kind: Kind::MoveTo,
399                 target: Some(pc),
400                 value: 70 * BAND - (steps + chop_t) - race_pen - deny_pen + pbonus,
401             });
402         }
403     }
404     // anti-starvation fell anything (band 30)
405     for p in state.trees.iter().filter(|p| p.size >= 1) {
406         let pc = p.pos();
407         if !d.contains_key(&pc) {
408             continue;
409         }
410         if hoard && !threatened(pc) {
411             continue; // Hoard: no fells unless the tree is under denial threat
412         }
413         let steps = eta(&d, pc, ms);
414         let chop_t = ((p.health + u.chop_power.max(1) - 1) / u.chop_power.max(1)) as i64;
415         let race_pen = match race(pc, steps) {
416             None => continue, // doomed: they fell it before we arrive - skip, don't donate the travel
417             Some(pen) => pen,
418         };
419         if pc == u.pos() {
420             out.push(Cand {
421                 kind: Kind::ChopHere,
422                 target: Some(pc),
423                 value: 31 * BAND - chop_t - race_pen,
424             });
425         } else {
426             out.push(Cand {
427                 kind: Kind::MoveTo,
428                 target: Some(pc),
429                 value: 30 * BAND - (steps + chop_t) - race_pen,
430             });
431         }
432     }
433     // partial bank / park (band 10)

```

```

434 out.push(Cand {
435   kind: if u.total_carried() > 0 {
436     Kind::Bank
437   } else {
438     Kind::Park
439   },
440   target: None,
441   value: 10 * BAND,
442 });
443 } else {
444   // `plant_cell` is hoisted above (before band 95/80) so band 80 can share it too -
445   // see its doc comment there. Bands 88 (below) and 50/49 (further down) just consume
446   // it.
447   // 1) plant carried banana (band 88) at the best free base cell
448   if u.carry[BANANA] > 0 {
449     if let Some(tc) = plant_cell {
450       let kind = if u.pos() == tc {
451         Kind::PlantHere
452       } else {
453         Kind::MoveTo
454       };
455       out.push(Cand {
456         kind,
457         target: Some(tc),
458         value: 88 * BAND - eta(&d, tc, ms),
459       });
460     }
461   }
462   // 3) standing on a ripe wanted fruit (band 75)
463   if let Some(p) = state.trees.iter().find(|p| p.pos() == u.pos()) {
464     if p.fruits > 0 && u.harvest_power > 0 && u.free_capacity() > 0 {
465       let ty = ge_fruit_ty(&p.tree_type);
466       let funding = plan.want_chopper || plan.want_feeder;
467       let want = (funding && ty.map_or(false, |t| t < 3 && plan.need_fund[t]))
468         || (!plan.want_chopper
469           && (p.tree_type == "BANANA"
470             || (p.tree_type == "APPLE"
471                 && state
472                   .water_cells
473                   .iter()
474                   .any(|w| manhattan(*w, p.pos()) == 1))))
475         || plan.phase == Phase::Hoard; // Hoard wants EVERYTHING ripe standing under foot too
476       if want {
477         out.push(Cand {
478           kind: Kind::Harvest,
479           target: Some(u.pos()),
480           value: 75 * BAND,
481         });
482       }
483     }
484   }
485   // B2 (Hoard): wallet-building - travel to ANY ripe fruit tree. Fruit is points AND
486   // wallet fuel during Hoard, so there is no per-type targeting like the funding/printer
487   // bands below (those stay as-is; the matcher just takes the max of every band pushed).
488   if plan.phase == Phase::Hoard {
489     for p in state
490       .trees
491       .iter()
492       .filter(|p| p.fruits > 0 && d.contains_key(&p.pos()))
493     {
494       let pc = p.pos();
495       out.push(Cand {
496         kind: Kind::MoveTo,
497         target: Some(pc),
498         value: 62 * BAND - eta(&d, pc, ms),
499       });
500     }
501   }
502   // 4) FUNDING (bands 60/58) - for the chopper OR a pending 3rd hand (R6b.2: the old
503   // feeder never trained because post-funding nobody harvested plum/lemon/apple)
504   if plan.want_chopper || plan.want_feeder {
505     // v1.28.1: the chopper is EXISTENTIAL (60/58) but a 3rd hand is a LUXURY - its
506     // funding (45/44) must never displace printer/seed work (50/48). The v1.28.0
507     // regression: perpetual feeder-funding starved the farm on lemon-poor maps.
508     let (fund_hi, fund_lo) = if plan.want_chopper {
509       (60, 58)
510     } else {
511       (45, 44)
512     };
513     // Gatekeeper verdict #3 (post-b14ebc7) fixed two compounding defects in one change:
514     // (a) BAND COLLISION - e09ac48 (iron, 64/63) and b14ebc7 (fruit, 63) independently
515     // landed on the SAME band, 63, so a troll needing both at once (routine: the
516     // ladder's last hand needs all four resources together) picked whichever was
517     // physically closer instead of the scarcer one. Iron has no fruit-harvest
518     // alternative (B2.1: "iron is scarce and un-substitutable") so it must win
519     // unconditionally - bumped to 65/64, strictly above the fruit band (63).
520     // (b) T_SWITCH CLIFF - all these bands were gated `phase == Phase::Hoard` only, so
521     // at t=140 a nearly-complete wallet was abandoned instantly (funding fell to

```

```

522 // fund_lo=44, below Printer's 48/50) and the ladder's last hand never trained
523 // (chopper 1/14 games). The elevated bands extend through a grace window scoped to
524 // `want_feeder` (the ladder is still incomplete) instead of Hoard alone – it covers
525 // Hoard (want_feeder is true throughout the ladder) AND the Factory grace (want_feeder
526 // stays true until the ladder finishes), self-extinguishing once the ladder completes
527 // (n reaches GE_MAX_TROLLS).
528 // v1.35.0 (T-hand): renamed `scale_funding` -> `ladder_funding` and DROPPED the
529 // `plan.phase != Phase::Tempo` gate – the elevated funding stack now serves ANY
530 // pending ladder hand, including Tempo's revived 3rd hand (GE_MAX_TROLLS 2->3), not
531 // just Scale's Hoard/Factory ladder. Graceful: a MoveTo/Mine candidate only exists
532 // where ripe deficit fruit / adjacent iron actually exists on the map, and
533 // `want_feeder` self-extinguishes the instant the pending hand trains – so Tempo
534 // degrades to today's champion behavior on any map where the wallet never fills.
535 // The generic wallet band (62, ~line 207 above) is UNTOUCHED: it stays gated on
536 // `plan.phase == Phase::Hoard` directly, never on this variable.
537 let ladder_funding = plan.want_feeder;
538 if plan.need_iron && u.chop_power > 0 {
539   if state
540     .iron_cells
541     .iter()
542     .any(|ic| manhattan(u.pos(), *ic) == 1)
543   {
544     let v = if ladder_funding { 65 } else { fund_hi };
545     out.push(Cand {
546       kind: Kind::Mine,
547       target: Some(u.pos()),
548       value: v * BAND,
549     });
550   } else if let Some(c) = state
551     .iron_cells
552     .iter()
553     .flat_map(|ic| ortho_neighbors(*ic))
554     .filter(|c| d.contains_key(c))
555     .min_by_key(|c| (d[c], tie_mix(*c, salt)))
556   {
557     let v = if ladder_funding { 64 } else { fund_hi };
558     out.push(Cand {
559       kind: Kind::MoveTo,
560       target: Some(c),
561       value: v * BAND - eta(&d, c, ms),
562     });
563   }
564 }
565 // Deficit-fruit funding (PLUM/LEMON/APPLE): same grace window, one band below iron.
566 let fruit_band = if ladder_funding { 63 } else { fund_lo };
567 for p in state.trees.iter().filter(|p| {
568   p.fruits > 0
569   && d.contains_key(&p.pos())
570   && ge_fruit_ty(&p.tree_type).map_or(false, |t| t < 3 && plan.need_fund[t])
571 }) {
572   let pc = p.pos();
573   out.push(Cand {
574     kind: Kind::MoveTo,
575     target: Some(pc),
576     value: fruit_band * BAND - eta(&d, pc, ms),
577   });
578 }
579 }
580 // 5) PRINTER (bands 52/50/49) – v1.37.0-nanaflow (user replay finding #2): TREE-FIRST.
581 // Harvesting a ripe seed tree directly converts its fruit straight into a farm seed;
582 // banked tent stock is just as harvestable a turn later. So a ripe seed tree now
583 // outranks the tent unconditionally (band 52, the old `inv[BANANA] == 0` gate is
584 // REMOVED – harvested even with tent stock on hand). PICK/park (50/49, unchanged) is
585 // the fallback once no ripe seed tree is reachable; excess bananas accumulate in the
586 // tent via the existing full->bank flow (1pt banked each, or 8pt later via plant->fell).
587 if plan.base_trees < plan.farm_cap {
588   for p in state.trees.iter().filter(|p| {
589     p.fruits > 0
590     && d.contains_key(&p.pos())
591     && (p.tree_type == "BANANA"
592       || (p.tree_type == "APPLE"
593         && state
594           .water_cells
595           .iter()
596           .any(|w| manhattan(*w, p.pos()) == 1)))
597   }) {
598     let pc = p.pos();
599     out.push(Cand {
600       kind: Kind::MoveTo,
601       target: Some(pc),
602       value: 52 * BAND - eta(&d, pc, ms),
603     });
604   }
605   // v1.41.0-nopickloop: only PICK (or travel to pick) if a plantable cell
606   // actually exists (plant_cell.is_some()) – picking a banana with nowhere to
607   // plant it is pure waste that just re-parks the starter on a scarce cell.
608   if inv[BANANA] > 0 && u.free_capacity() > 0 && plant_cell.is_some() {
609     // target = shack: dedupes the pick errand across multiple hands (R6b.2)

```



```

610     if manhattan(u.pos(), shack) == 1 {
611         out.push(Cand {
612             kind: Kind::Pick,
613             target: Some(shack),
614             value: 50 * BAND,
615         });
616     } else {
617         out.push(Cand {
618             kind: Kind::Park,
619             target: Some(shack),
620             value: 50 * BAND - 1,
621         });
622     }
623 }
624 }
625 // 6) chop help (band 40) + anti-starvation (band 30)
626 if plan.starter_chop && u.chop_power > 0 {
627     for p in state
628         .trees
629         .iter()
630         .filter(|p| fell_ok(p) && own_half(p) && within_roam(p))
631     {
632         if u.free_capacity() == 0 {
633             break;
634         }
635         let pc = p.pos();
636         if !d.contains_key(&pc) {
637             continue;
638         }
639         if hoard && !threatened(pc) {
640             continue; // Hoard: no fells unless the tree is under denial threat
641         }
642         let steps = eta(&d, pc, ms);
643         let chop_t = ((p.health + u.chop_power.max(1) - 1) / u.chop_power.max(1)) as i64;
644         let race_pen = match race(pc, steps) {
645             None => continue, // doomed: they fell it before we arrive – skip, don't donate the travel
646             Some(pen) => pen,
647         };
648         let pbonus = pressure_bonus(pc, race_pen);
649         if pc == u.pos() {
650             out.push(Cand {
651                 kind: Kind::ChopHere,
652                 target: Some(pc),
653                 value: 42 * BAND - chop_t - race_pen + pbonus,
654             });
655         } else {
656             out.push(Cand {
657                 kind: Kind::MoveTo,
658                 target: Some(pc),
659                 value: 40 * BAND - (steps + chop_t) - race_pen + pbonus,
660             });
661         }
662     }
663     if u.free_capacity() > 0 {
664         for p in state.trees.iter().filter(|p| p.size >= 1) {
665             let pc = p.pos();
666             if !d.contains_key(&pc) {
667                 continue;
668             }
669             if hoard && !threatened(pc) {
670                 continue; // Hoard: no fells unless the tree is under denial threat
671             }
672             let steps = eta(&d, pc, ms);
673             let chop_t =
674                 ((p.health + u.chop_power.max(1) - 1) / u.chop_power.max(1)) as i64;
675             let race_pen = match race(pc, steps) {
676                 None => continue, // doomed: they fell it before we arrive – skip, don't donate the travel
677                 Some(pen) => pen,
678             };
679             if pc == u.pos() {
680                 out.push(Cand {
681                     kind: Kind::ChopHere,
682                     target: Some(pc),
683                     value: 31 * BAND - chop_t - race_pen,
684                 });
685             } else {
686                 out.push(Cand {
687                     kind: Kind::MoveTo,
688                     target: Some(pc),
689                     value: 30 * BAND - (steps + chop_t) - race_pen,
690                 });
691             }
692         }
693     }
694 }
695 // 6.5) IDLE-FRUIT (band 38, design D1 – champion loss taxonomy 2026-07-08 morning,
696 // docs/silver-experiment-log.md: 45% of all losses are opponents out-fruiting us,
697 // HARVEST+DROP 91-307 vs our flat 20-90). Strictly ABOVE anti-starvation (31/30) –

```

```

698 // never competes with keeping the wood supply alive – and strictly BELOW chop-help
699 // (42/40) and every printer/funding band above it (52/50/49/48/45/44/63/64/65/60/58) –
700 // this is the fix for the v1.24.0-fruitbank trap (arena -1.0), which ranked
701 // fruit-chasing ABOVE chop-help and lost. Because every one of those higher bands
702 // already claims its own trees first, band 38 only ever wins the joint assignment on
703 // a turn where nothing more valuable was available – it converts an otherwise-idle
704 // turn into fruit points and never displaces wood work, seed work, or funding. No
705 // per-type/own-half/roam gating on purpose ("harvest ANY ripe fruit"); mirrors the
706 // ChopHere/MoveTo split used by every other band in this function.
707 if u.harvest_power > 0 && u.free_capacity() > 0 {
708     for p in state
709         .trees
710         .iter()
711         .filter(|p| p.fruits > 0 && d.contains_key(&p.pos()))
712     {
713         let pc = p.pos();
714         let steps = eta(&d, pc, ms);
715         // reviewer IMPORTANT follow-up: same race check every other tree-targeting band
716         // uses (70/72, 40/42, 30/31) – an enemy chopper already standing on this tree
717         // fells it before we arrive, so chasing it donates the travel just like the
718         // wood-fell case (doomed-target chasing). Unlike those bands, a joinable race
719         // (Some(pen)) does NOT subtract the share-penalty: sharing a cell with an enemy
720         // CHOPPER while WE harvest fruit isn't a wood-split situation (apply_chop's
721         // round-robin split is a wood-only mechanic) – Some(_) here only means "not
722         // doomed"; a same-cell Harvest (steps=0) is never doomed in practice (their_turns
723         // is 0 only if the tree's health is already 0, which cannot coexist with the
724         // `p.fruits > 0` filter above), so this uniform pre-branch check costs it nothing.
725         if race(pc, steps).is_none() {
726             continue; // doomed: they fell it before we arrive – skip, don't donate the travel
727         }
728         if pc == u.pos() {
729             out.push(Cand {
730                 kind: Kind::Harvest,
731                 target: Some(pc),
732                 value: 38 * BAND,
733             });
734         } else {
735             out.push(Cand {
736                 kind: Kind::MoveTo,
737                 target: Some(pc),
738                 value: 38 * BAND - steps,
739             });
740         }
741     }
742 }
743 // fallback (band 10)
744 out.push(Cand {
745     kind: if u.total_carried() > 0 {
746         Kind::Bank
747     } else {
748         Kind::Park
749     },
750     target: None,
751     value: 10 * BAND,
752 });
753 }
754
755 // stickiness: prefer last turn's target on near-ties (see STICKY)
756 let last = LAST_TGT.with(|m| m.borrow().get(&u.id).copied());
757 if let Some(lt) = last {
758     for c in out.iter_mut() {
759         if c.target == Some(lt) {
760             c.value += STICKY;
761         }
762     }
763 }
764 // canonical order + cap: by (-value, target) – never by discovery order
765 out.sort_by_key(|c| (-c.value, c.target));
766 out.truncate(K);
767 out
768 }
769
770 fn select_assignments(state: &State, plan: &Plan, my: &[Troll]) -> Assignments {
771     let salt = tie_salt(state);
772     let mut ids: Vec<i32> = my.iter().map(|t| t.id).collect();
773     ids.sort();
774     let trolls: Vec<&Troll> = ids
775         .iter()
776         .map(|id| my.iter().find(|t| t.id == *id).unwrap())
777         .collect();
778     let cands: Vec<Vec<Cand>> = trolls
779         .iter()
780         .map(|t| candidates(state, plan, my, t, salt))
781         .collect();
782     let claim_infos: Vec<Vec<Option<ClaimInfo>>> = trolls
783         .iter()
784         .zip(cands.iter())
785         .map(|(t, cs)| {

```

```

786     let d = bfs_distances(&state.walkable, &[t.pos()]);
787     cs.iter()
788         .map(|c| {
789             let steps = c.target.map_or(0, |tc| eta(&d, tc, t.movement_speed));
790             claim_info(state, c, steps)
791         })
792         .collect()
793 })
794 .collect();
795
796 let n = ids.len();
797 let mut best: Option<(i64, Vec<usize>)> = None;
798 let mut pick = vec![0; n];
799 if n > 0 {
800     loop {
801         let mut ok = true;
802         for i in 0..n {
803             for j in 0..i {
804                 if let (Some(a), Some(b)) = (claim_infos[i][pick[i]], claim_infos[j][pick[j]]) {
805                     if claims_conflict(a, b) {
806                         ok = false;
807                         break;
808                     }
809                 }
810             }
811             if !ok {
812                 break;
813             }
814         }
815         if ok {
816             let total: i64 = (0..n).map(|i| cands[i][pick[i]].value).sum();
817             let better = match &best {
818                 None => true,
819                 Some((bt, bp)) => total > *bt || (total == *bt && pick < *bp),
820             };
821             if better {
822                 best = Some((total, pick.clone()));
823             }
824         }
825         let mut i = 0;
826         loop {
827             if i == n {
828                 break;
829             }
830             pick[i] += 1;
831             if pick[i] < cands[i].len() {
832                 break;
833             }
834             pick[i] = 0;
835             i += 1;
836         }
837         if i == n {
838             break;
839         }
840     }
841 }
842
843 Assignments {
844     ids,
845     cands,
846     picks: best.map(|(_, picks)| picks).unwrap_or_default(),
847 }
848 }
849
850 fn render_assignments(
851     state: &State,
852     plan: &Plan,
853     my: &[Troll],
854     assignments: &Assignments,
855     update_last_target: bool,
856 ) -> HashMap<i32, String> {
857     // render (troll-id order; camp-cell claiming stays deterministic via claimed_drop)
858     let mut cmd_by_id: HashMap<i32, String> = HashMap::new();
859     let mut claimed_drop: HashSet<Cell> = HashSet::new();
860     if !assignments.picks.is_empty() {
861         for (i, id) in assignments.ids.iter().enumerate() {
862             let u = my.iter().find(|t| t.id == *id).unwrap();
863             let d = bfs_distances(&state.walkable, &[u.pos()]);
864             let c = &assignments.cands[i][assignments.picks[i]];
865             if update_last_target {
866                 if let (Kind::MoveTo, Some(tc)) = (&c.kind, c.target) {
867                     LAST_TGT.with(|m| {
868                         let mut m = m.borrow_mut();
869                         if let Some(prev) = m.get(id) {
870                             if *prev != tc && u.pos() != *prev {
871                                 FLAPS.with(|f| *f.borrow_mut() += 1);
872                             }
873                         }
874                     })
875                 }
876             }
877         }
878     }
879 }

```

```

874         m.insert(*id, tc);
875     });
876 } else {
877     LAST_TGT.with(|m| m.borrow_mut().remove(id));
878 }
879 }
880 let cmd = match (&c.kind, c.target) {
881     (Kind::Bank, _) => motion::bank_cmd(state, plan.shack, u, &d, &mut claimed_drop,
882     // idle band-10 (target None) gets the ring-2-aware scarce-camp step-back;
883     // the band-49 park-to-pick ERRAND (target Some(shack)) never does – it is
884     // goal-directed (must reach manhattan==1 to unlock PICK) and the ring-2
885     // redirect has no such convergence guarantee (reviewer CRITICAL fix, see
886     // motion::park_cmd's doc comment).
887     (Kind::Park, park_target) => motion::park_cmd(
888         state,
889         plan.shack,
890         u,
891         &d,
892         &mut claimed_drop,
893         park_target.is_none(),
894     ),
895     (Kind::ChopHere, _) => format!("CHOP {}", u.id),
896     (Kind::PlantHere, _) => format!("PLANT {} BANANA", u.id),
897     (Kind::Harvest, _) => format!("HARVEST {}", u.id),
898     (Kind::Mine, _) => format!("MINE {}", u.id),
899     (Kind::Pick, _) => format!("PICK {} BANANA", u.id),
900     (Kind::MoveTo, Some(tc)) => format!("MOVE {} {} {}", u.id, tc.0, tc.1),
901     (Kind::MoveTo, None) => format!("MOVE {} {} {}", u.id, plan.shack.0, plan.shack.1),
902 };
903 cmd_by_id.insert(*id, cmd);
904 }
905 }
906 cmd_by_id
907 }
908
909 fn move_intents(cmd_by_id: &HashMap<i32, String>) -> Vec<(i32, Cell)> {
910     let mut intents: Vec<(i32, Cell)> = cmd_by_id
911         .iter()
912         .filter_map(|(id, c)| {
913             let p: Vec<&str> = c.split_whitespace().collect();
914             if p.len() == 4 && p[0] == "MOVE" {
915                 Some((*id, (p[2].parse().ok()?, p[3].parse().ok()?)))
916             } else {
917                 None
918             }
919         })
920         .collect();
921     intents.sort();
922     intents
923 }
924
925 fn pin_landing(my: &[Troll], cmd_by_id: &mut HashMap<i32, String>, landing: HashMap<i32, Cell>) {
926     for (id, cell) in landing {
927         let cur = my.iter().find(|t| t.id == id).map(|t| t.pos());
928         if cur != Some(cell) {
929             cmd_by_id.insert(id, format!("MOVE {} {} {}", id, cell.0, cell.1));
930         }
931     }
932 }
933
934 fn best_progress_without_stationary(
935     state: &State,
936     mover: &Troll,
937     goal: Cell,
938     stationary: &HashSet<Cell>,
939 ) -> Option<i32> {
940     let dg = bfs_distances(&state.walkable, &[goal]);
941     let dp = bfs_distances(&state.walkable, &[mover.pos()]);
942     let here = *dg.get(&mover.pos())?;
943     let mut best = 0;
944     for c in &state.walkable {
945         if stationary.contains(c) {
946             continue;
947         }
948         let Some(&from_here) = dp.get(c) else {
949             continue;
950         };
951         if from_here == 0 || from_here > mover.movement_speed {
952             continue;
953         }
954         let Some(&to_goal) = dg.get(c) else {
955             continue;
956         };
957         let progress = here - to_goal;
958         if progress >= 0 {
959             best = best.max(progress);
960         }
961     }
962 }

```

```

962     Some(best)
963 }
964
965 fn blocker_landing_progress(
966     state: &State,
967     mover: &Troll,
968     goal: Cell,
969     blocker_cell: Cell,
970 ) -> Option<i32> {
971     if !state.walkable.contains(&blocker_cell) {
972         return None;
973     }
974     let dg = bfs_distances(&state.walkable, &[goal]);
975     let dp = bfs_distances(&state.walkable, &[mover.pos()]);
976     let here = *dg.get(&mover.pos())?;
977     let from_here = *dp.get(&blocker_cell)?;
978     if from_here == 0 || from_here > mover.movement_speed {
979         return None;
980     }
981     let progress = here - *dg.get(&blocker_cell)?;
982     (progress > 0).then_some(progress)
983 }
984
985 fn candidate_conflicts(
986     assignments: &Assignments,
987     blocker_idx: usize,
988     target: Option<Cell>,
989 ) -> bool {
990     let Some(target) = target else {
991         return false;
992     };
993     assignments.ids.iter().enumerate().any(|(i, _)| {
994         i != blocker_idx && assignments.cands[i][assignments.picks[i]].target == Some(target)
995     })
996 }
997
998 fn candidate_can_move_for_yield(plan: &Plan, u: &Troll, cand: &Cand) -> bool {
999     match cand.kind {
1000         Kind::MoveTo | Kind::Park => true,
1001         Kind::Bank => manhattan(u.pos(), plan.shack) != 1,
1002         Kind::ChopHere | Kind::PlantHere | Kind::Harvest | Kind::Mine | Kind::Pick => false,
1003     }
1004 }
1005
1006 fn reselect_blocker_for_yield(
1007     plan: &Plan,
1008     my: &[Troll],
1009     assignments: &mut Assignments,
1010     blocker_id: i32,
1011 ) -> bool {
1012     let Some(blocker_idx) = assignments.idx(blocker_id) else {
1013         return false;
1014     };
1015     let Some(u) = my.iter().find(|t| t.id == blocker_id) else {
1016         return false;
1017     };
1018     let old_pick = assignments.picks[blocker_idx];
1019     let old_target = assignments.cands[blocker_idx][old_pick].target;
1020     for new_pick in 0..assignments.cands[blocker_idx].len() {
1021         if new_pick == old_pick {
1022             continue;
1023         }
1024         let cand = &assignments.cands[blocker_idx][new_pick];
1025         if old_target.is_some() && cand.target == old_target {
1026             continue;
1027         }
1028         if candidate_conflicts(assignments, blocker_idx, cand.target) {
1029             continue;
1030         }
1031         if !candidate_can_move_for_yield(plan, u, cand) {
1032             continue;
1033         }
1034         assignments.picks[blocker_idx] = new_pick;
1035         return true;
1036     }
1037     false
1038 }
1039
1040 fn yield_pass(
1041     state: &State,
1042     plan: &Plan,
1043     my: &[Troll],
1044     assignments: &mut Assignments,
1045     intents: &[(i32, Cell)],
1046     landing: &HashMap<i32, Cell>,
1047 ) -> bool {
1048     #[derive(Clone)]
1049     struct YieldCandidate {

```

```

1050     mover_id: i32,
1051     blocker_id: i32,
1052     mover_value: i64,
1053     blocker_value: i64,
1054 }
1055
1056 let moving: HashSet<i32> = intents.iter().map(|(id, _)| *id).collect();
1057 let stationary_cells: HashSet<Cell> = my
1058     .iter()
1059     .filter(|t| !moving.contains(&t.id))
1060     .map(|t| t.pos())
1061     .collect();
1062 let stationary: Vec<&Troll> = my.iter().filter(|t| !moving.contains(&t.id)).collect();
1063 let mut pairs: Vec<YieldCandidate> = Vec::new();
1064
1065 for (mover_id, goal) in intents {
1066     let Some(mover) = my.iter().find(|t| t.id == *mover_id) else {
1067         continue;
1068     };
1069     if landing.get(mover_id) != Some(&mover.pos()) {
1070         continue;
1071     }
1072     let Some(mover_value) = assignments.selected_value(*mover_id) else {
1073         continue;
1074     };
1075     let Some(normal_best) =
1076         best_progress_without_stationary(state, mover, *goal, &stationary_cells)
1077     else {
1078         continue;
1079     };
1080     if normal_best > 0 {
1081         continue;
1082     }
1083     for blocker in &stationary {
1084         let Some(blocker_value) = assignments.selected_value(blocker.id) else {
1085             continue;
1086         };
1087         if mover_value <= blocker_value {
1088             continue;
1089         }
1090         let Some(blocked_progress) =
1091             blocker_landing_progress(state, mover, *goal, blocker.pos())
1092         else {
1093             continue;
1094         };
1095         if blocked_progress > normal_best {
1096             pairs.push(YieldCandidate {
1097                 mover_id: *mover_id,
1098                 blocker_id: blocker.id,
1099                 mover_value,
1100                 blocker_value,
1101             });
1102         }
1103     }
1104 }
1105
1106 pairs.sort_by_key(|p| (-p.mover_value, p.blocker_value, p.mover_id, p.blocker_id));
1107 for p in pairs {
1108     let mut trial = assignments.clone();
1109     if reselect_blocker_for_yield(plan, my, &mut trial, p.blocker_id) {
1110         *assignments = trial;
1111         if DEBUG {
1112             eprintln!(
1113                 "@TFYIELD t={ } blocker={ } mover={ }",
1114                 state.turn, p.blocker_id, p.mover_id
1115             );
1116         }
1117         return true;
1118     }
1119 }
1120 false
1121 }
1122
1123 /// Joint assignment: exhaustive over per-troll top-K candidates, maximize total value,
1124 /// conflicting target claims forbidden, ties broken by the lexicographic pick vector.
1125 pub fn assign(state: &State, plan: &Plan, my: &[Troll]) -> HashMap<i32, String> {
1126     let assignments = select_assignments(state, plan, my);
1127     render_assignments(state, plan, my, &assignments, true)
1128 }
1129
1130 /// Assignment plus the live L3 motion pass. This keeps all task economics in the
1131 /// joint matcher, then lets one lower-value stationary teammate yield to an urgent
1132 /// blocked mover before final MOVE landing cells are pinned.
1133 pub fn assign_resolved(state: &State, plan: &Plan, my: &[Troll]) -> HashMap<i32, String> {
1134     let mut assignments = select_assignments(state, plan, my);
1135     let initial_cmds = render_assignments(state, plan, my, &assignments, false);
1136     let initial_intents = move_intents(&initial_cmds);
1137     let initial_landing = motion::solve_moves(state, my, &initial_intents);

```

```

1138
1139     let yielded = yield_pass(
1140         state,
1141         plan,
1142         my,
1143         &mut assignments,
1144         &initial_intents,
1145         &initial_landing,
1146     );
1147     let mut cmd_by_id = render_assignments(state, plan, my, &assignments, true);
1148     let landing = if yielded {
1149         let intents = move_intents(&cmd_by_id);
1150         motion::solve_moves(state, my, &intents)
1151     } else {
1152         initial_landing
1153     };
1154     pin_landing(my, &mut cmd_by_id, landing);
1155     cmd_by_id
1156 }

```

5 botmain/motion.rs

L3 motion — the joint MOVE solver: chooses landing cells jointly (swaps/chains), shuffle-invariant; watchdog.
(274 lines)

```
1  //! Motion layer (R3b): everything about *getting trolls where their job says without
2  //! wasting moves* — distinct camp-cell claiming for bank/park (v1.20.0, the #1 near-camp
3  //! block fix) and the anti-stall watchdog (sidestep after 2 stuck turns). Extracted
4  //! VERBATIM from decide_elite (closures → functions); behavior equality is enforced by
5  //! the black-box harness. The corridor tests (tests/motion_corridor.rs) pin the required
6  //! swap-pipeline behavior this layer must keep enabling.
7  use super::*;
8  use std::cell::RefCell;
9  use std::collections::{HashMap, HashSet};
10
11 thread_local! {
12     // anti-stall watchdog (mirrors decide_rhea/RH_LASTPOS):
13     // troll id → (x, y, same-pos streak while MOVEing). Reset at turn 1.
14     static GE_LASTPOS: RefCell<HashMap<i32, (i32, i32, u8)>> = RefCell::new(HashMap::new());
15 }
16
17 /// Turn-1 reset of the watchdog memory.
18 pub fn reset() {
19     GE_LASTPOS.with(|m| m.borrow_mut().clear());
20 }
21
22 /// nearest UNCLAIMED walkable drop cell (& claim it) — trolls heading to the camp claim
23 /// DISTINCT shack-adjacent cells so they don't converge on one cell and block each other.
24 pub fn pick_camp_cell(
25     state: &State,
26     shack: Cell,
27     d: &HashMap<Cell, i32>,
28     claimed: &mut HashSet<Cell>,
29 ) -> Cell {
30     let free = ortho_neighbors(shack)
31         .into_iter()
32         .filter(|c| state.walkable.contains(c) && !claimed.contains(c))
33         .min_by_key(|c| d.get(c).copied().unwrap_or(1 << 30));
34     let cell = free
35         .or_else(|| {
36             // all camp cells claimed (rare): fall back to the nearest walkable one
37             ortho_neighbors(shack)
38                 .into_iter()
39                 .filter(|c| state.walkable.contains(c))
40                 .min_by_key(|c| d.get(c).copied().unwrap_or(1 << 30))
41         })
42         .unwrap_or(shack);
43     claimed.insert(cell);
44     cell
45 }
46
47 /// DROP if shack-adjacent, else MOVE toward a claimed camp cell.
48 pub fn bank_cmd(
49     state: &State,
50     shack: Cell,
51     u: &Troll,
52     d: &HashMap<Cell, i32>,
53     claimed: &mut HashSet<Cell>,
54 ) -> String {
55     if manhattan(u.pos(), shack) == 1 {
56         format!("DROP {}", u.id)
57     } else {
58         let c = pick_camp_cell(state, shack, d, claimed);
59         format!("MOVE {} {} {} {}", u.id, c.0, c.1)
60     }
61 }
62
63 /// MOVE toward a claimed camp cell. `idle=true` (band-10 idle parking) additionally steps
64 /// back from a SCARCE camp — v1.41.0-nopickloop (user-observed): when the shack has <=2
65 /// walkable ortho-neighbors, an idle-parking troll piling onto the one or two cells a
66 /// banker actually needs blocks the bank. Prefer the nearest unclaimed, reachable
67 /// manhattan-2 ring cell instead (one step further out, out of the banker's way); fall
68 /// back to the normal camp-cell claim if no such cell is reachable/free (e.g. a true 1-2
69 /// cell dead end with nothing beyond it).
70 ///
71 /// `idle=false` (the band-49 park-to-pick ERRAND, planner.rs `Kind::Park` with
72 /// `target: Some(shack)` always takes the direct camp-cell approach and NEVER the ring-2
73 /// detour — reviewer-caught CRITICAL bug: the errand is GOAL-DIRECTED (it must reach
74 /// manhattan==1 to unlock band-50's PICK), but the ring-2 redirect has no convergence
75 /// guarantee toward that goal. `claimed` is a fresh `HashSet` every `assign()` call (see
76 /// planner.rs), so a redirected errand that reaches its own ring-2 cell sees, next turn,
77 /// that same cell as the nearest unclaimed manhattan-2 option (distance 0 from itself) and
78 /// reissues a MOVE to its own position forever — a permanent stall on scarce-camp maps that
79 /// the anti-stall watchdog can't catch (it only sidesteps a MOVE whose target differs from
80 /// the troll's current cell).
81 pub fn park_cmd(
82     state: &State,
```



```

83     shack: Cell,
84     u: &Troll,
85     d: &HashMap<Cell, i32>,
86     claimed: &mut HashSet<Cell>,
87     idle: bool,
88 ) -> String {
89     if idle {
90         let camp_cells = ortho_neighbors(shack)
91             .iter()
92             .filter(|c| state.walkable.contains(*c))
93             .count();
94         if camp_cells <= 2 {
95             let ring2 = state
96                 .walkable
97                 .iter()
98                 .filter(|c| manhattan(*c, shack) == 2 && !claimed.contains(*c))
99                 .filter_map(|c| d.get(c).map(|&dist| (*c, dist)))
100                 .min_by_key(|(c, dist)| (*dist, *c));
101             if let Some((c, _)) = ring2 {
102                 claimed.insert(c);
103                 return format!("MOVE {} {} {}", u.id, c.0, c.1);
104             }
105         }
106     }
107     let c = pick_camp_cell(state, shack, d, claimed);
108     format!("MOVE {} {} {}", u.id, c.0, c.1)
109 }
110
111 /// ANTI-STALL WATCHDOG: if a troll issued a MOVE but hasn't moved for 2+ consecutive
112 /// turns, it is self-blocked – sidestep to a free orthogonally-adjacent walkable cell.
113 /// This is the #1 arena loss cause (self-block stalls).
114 pub fn watchdog(state: &State, my: &[Troll], cmd_by_id: &mut HashMap<i32, String>) {
115     GE_LASTPOS.with(|cell| {
116         let mut m = cell.borrow_mut();
117         for t in my {
118             let cur = t.pos();
119             let is_move = cmd_by_id
120                 .get(&t.id)
121                 .map_or(false, |c| c.starts_with("MOVE "));
122             let entry = m.entry(t.id).or_insert((cur.0, cur.1, 0u8));
123             let stuck = entry.0 == cur.0 && entry.1 == cur.1;
124             entry.2 = if stuck && is_move {
125                 entry.2.saturating_add(1)
126             } else {
127                 0
128             };
129             entry.0 = cur.0;
130             entry.1 = cur.1;
131             let streak = entry.2;
132             if streak >= 2 && is_move {
133                 // parse the MOVE target; only sidestep if it isn't the cur cell
134                 let tgt = cmd_by_id.get(&t.id).and_then(|c| {
135                     let p: Vec<&str> = c.split_whitespace().collect();
136                     if p.len() == 4 {
137                         Some((p[2].parse:::<i32>().ok()?, p[3].parse:::<i32>().ok()?))
138                     } else {
139                         None
140                     }
141                 });
142                 if let Some((tx, ty)) = tgt {
143                     if (tx, ty) != cur {
144                         let mut cands: Vec<Cell> = Vec::new();
145                         for nb in ortho_neighbors(cur) {
146                             if state.walkable.contains(&nb) && !my.iter().any(|o| o.pos() == nb) {
147                                 cands.push(nb);
148                             }
149                         }
150                         if !cands.is_empty() {
151                             let pick = cands[(rh_rand() as usize) % cands.len()];
152                             cmd_by_id.insert(t.id, format!("MOVE {} {} {}", t.id, pick.0, pick.1));
153                             entry.2 = 0;
154                         }
155                     }
156                 }
157             }
158         }
159     });
160 }
161
162 // — R6a: JOINT MOVE SOLVER (the activity manager's motion stage) —————
163 // The sequential cascade let iteration order + tie-breaks decide who moves where.
164 // This solver takes ALL movement intents (troll -> goal cell) and chooses this turn's
165 // landing cells JOINTLY: maximize total progress toward goals under the verified engine
166 // rules (final-cell conflicts; adjacent cross-steps SWAP; vacated-cell chains resolve;
167 // stationary teammates are hard obstacles). Design criterion: SHUFFLE INVARIANCE – the
168 // result is a function of the objective only (canonical candidate order + exhaustive
169 // joint search + total-order tie-break), never of input order.
170

```

```

171 // Jointly choose this turn's MOVE landing cell per intent (troll id -> goal cell).
172 // Returns id -> landing cell (may be the troll's own cell = effectively WAIT/stay).
173 pub fn solve_moves(state: &State, my: &[Troll], intents: &[(i32, Cell)]) -> HashMap<i32, Cell> {
174     let moving: HashSet<i32> = intents.iter().map(|(id, _)| *id).collect();
175     let stationary: HashSet<Cell> = my
176         .iter()
177         .filter(|t| !moving.contains(&t.id))
178         .map(|t| t.pos())
179         .collect();
180
181     // canonical processing order: by troll id (input order must not matter)
182     let mut intents: Vec<(i32, Cell)> = intents.to_vec();
183     intents.sort();
184
185     // per troll: candidate landing cells within movement range, canonical order
186     let mut ids: Vec<i32> = Vec::new();
187     let mut cands: Vec<Vec<Cell, i32>> = Vec::new(); // (landing, progress toward goal)
188     for (id, goal) in &intents {
189         let t = match my.iter().find(|t| t.id == *id) {
190             Some(t) => t,
191             None => continue,
192         };
193         let dg = bfs_distances(&state.walkable, &[*goal]);
194         let dp = bfs_distances(&state.walkable, &t.pos());
195         let here = match dg.get(&t.pos()) {
196             Some(&d) => d,
197             None => {
198                 // goal unreachable: stay put (the watchdog / next replan handles it)
199                 ids.push(*id);
200                 cands.push(vec![(t.pos(), 0)]);
201                 continue;
202             }
203     };
204     let mut cs: Vec<(Cell, i32)> = state
205         .walkable
206         .iter()
207         .filter(|c| {
208             dp.get(*c)
209                 .map_or(false, |&d| d > 0 && d <= t.movement_speed)
210         })
211         .filter(|c| !stationary.contains(*c))
212         .filter_map(|c| dg.get(c).map(|&d| (*c, here - d)))
213         .filter(|(_, pr)| *pr >= 0) // progress or lateral sidestep; never retreat
214         .collect();
215     cs.push((t.pos(), 0)); // staying is always an option
216     cs.sort_by_key(|(c, pr)| (-pr, *c)); // canonical: best progress, then cell order
217     cs.truncate(8);
218     ids.push(*id);
219     cands.push(cs);
220 }
221
222 // exhaustive joint choice over  $\leq 8^n$  combos ( $n \leq \sim 4$  trolls): maximize total progress;
223 // validity = pairwise-distinct landing cells (swaps/chains through MOVING teammates are
224 // legal under the engine; stationary cells were excluded above). Ties -> lexicographic
225 // landing vector (one canonical rule; shuffle invariance holds).
226 let n = ids.len();
227 let mut best: Option<(i32, Vec<Cell>)> = None;
228 let mut pick = vec![0usize; n];
229 loop {
230     let landing: Vec<Cell> = (0..n).map(|i| cands[i][pick[i]].0).collect();
231     let distinct = {
232         let mut s: Vec<Cell> = landing.clone();
233         s.sort();
234         s.windows(2).all(|w| w[0] != w[1])
235     };
236     if distinct {
237         let total: i32 = (0..n).map(|i| cands[i][pick[i]].1).sum();
238         let better = match &best {
239             None => true,
240             Some((bt, bl)) => total > *bt || (total == *bt && landing < *bl),
241         };
242         if better {
243             best = Some((total, landing));
244         }
245     }
246     // odometer over candidate indices
247     let mut i = 0;
248     loop {
249         if i == n {
250             break;
251         }
252         pick[i] += 1;
253         if pick[i] < cands[i].len() {
254             break;
255         }
256         pick[i] = 0;
257         i += 1;
258     }
}

```

```

259         if i == n {
260             break;
261         }
262         if n == 0 {
263             break;
264         }
265     }
266
267     let mut out = HashMap::new();
268     if let Some( (_, landing)) = best {
269         for (i, id) in ids.iter().enumerate() {
270             out.insert(*id, landing[i]);
271         }
272     }
273     out
274 }

```

6 botmain/ownership.rs

Ownership pressure telemetry (@TFOWN/@TFPRESS) — the parked total-map-value-ownership diagnostic.
(426 lines)

```
1  /// Total-map value ownership diagnostic + pressure governor (v1.53.0-pressurefarm).
2  ///
3  /// `analyze`/`classify_tree`/`Ownership` below are the ORIGINAL DEBUG-only diagnostic
4  /// (unchanged): a rough, auditable ownership split from the live per-turn `State`, printed
5  /// only when `DEBUG` is enabled, never read by behavior.
6  ///
7  /// `assess`/`Pressure`/`PressureState` are the NEW pressure governor (Task 1): they DO feed
8  /// `tactics::plan_impl` every turn (unconditionally, not gated by DEBUG) so `Plan` can carry
9  /// a live pressure verdict into `planner.rs`. This is a deliberate, narrow exception to the
10 /// "diagnostic only" rule above — see docs/superpowers/plans/2026-07-09-pressurefarm-
11 /// ownership-score.md and docs/pressure-aware-farm.md. `assess` reuses the existing
12 /// `analyze`/`classify_tree`/`is_created_farm_tree` helpers verbatim (no behavior change to
13 /// them); it only adds a second, cheap pass over created-farm trees (bounded by farm size,
14 /// not map size) to classify which ones are exposed.
15 use super::tactics::Plan;
16 use super::*;
17 use std::cell::RefCell;
18 use std::collections::HashSet;
19
20 pub const OWN_MARGIN_TURNS: i32 = 3;
21 pub const OWN_FUTURE_SEED_VALUE: i32 = 1;
22 pub const OWN_CREATED_NEAR_TENT_R: i32 = 2;
23
24 const INF: i32 = 1_000_000;
25
26 thread_local! {
27     static INITIAL_TREES: RefCell<HashSet<(Cell, String)>> = RefCell::new(HashSet::new());
28     static INITIAL_READY: RefCell<bool> = RefCell::new(false);
29     static CFG_PRINTED: RefCell<bool> = RefCell::new(false);
30     static PRESS_CFG_PRINTED: RefCell<bool> = RefCell::new(false);
31 }
32
33 #[derive(Clone, Copy, Debug, Default, PartialEq, Eq)]
34 pub struct Ownership {
35     pub total: i32,
36     pub ours: i32,
37     pub opp: i32,
38     pub uncertain: i32,
39     pub dead: i32,
40     pub created_exposed: i32,
41     pub own_half_exposed: i32,
42 }
43
44 #[derive(Clone, Copy, Debug, PartialEq, Eq)]
45 enum Bucket {
46     Ours,
47     Opponent,
48     Uncertain,
49     Dead,
50 }
51
52 pub fn reset() {
53     INITIAL_TREES.with(|s| s.borrow_mut().clear());
54     INITIAL_READY.with(|r| *r.borrow_mut() = false);
55     CFG_PRINTED.with(|p| *p.borrow_mut() = false);
56     PRESS_CFG_PRINTED.with(|p| *p.borrow_mut() = false);
57 }
58
59 pub fn log(state: &State, plan: &Plan) {
60     ensure_initial(state);
61     if state.turn == 1 {
62         CFG_PRINTED.with(|printed| {
63             let mut printed = printed.borrow_mut();
64             if !*printed {
65                 eprintln!(
66                     "@TFOWNCFG margin={} future_seed={} created_near_tent_r={} farm_r=",
67                     OWN_MARGIN_TURNS, OWN_FUTURE_SEED_VALUE, OWN_CREATED_NEAR_TENT_R, plan.farm_r
68                 );
69                 *printed = true;
70             }
71         });
72     }
73     if !should_emit(state.turn) {
74         return;
75     }
76     let own = analyze(state, plan);
77     eprintln!(
78         "@TFOWN t={} total={} ours={} opp={} uncertain={} dead={} created_exposed={} own_half_exposed=",
79         state.turn,
80         own.total,
81         own.ours,
82         own.opp,
```

```

83     own.uncertain,
84     own.dead,
85     own.created_exposed,
86     own.own_half_exposed
87 );
88 }
89
90 pub fn analyze(state: &State, plan: &Plan) -> Ownership {
91     ensure_initial(state);
92
93     let opp_d = bfs_distances(&state.walkable, &[state.opp_shack]);
94     let mut out = Ownership::default();
95
96     for tree in &state.trees {
97         let value = tree_value(tree);
98         if value <= 0 {
99             continue;
100         }
101
102         let bucket = classify_tree(state, tree);
103         out.total += value;
104         match bucket {
105             Bucket::Ours => out.ours += value,
106             Bucket::Opponent => out.opp += value,
107             Bucket::Uncertain => out.uncertain += value,
108             Bucket::Dead => out.dead += value,
109         }
110
111         if is_created_farm_tree(tree, plan)
112             && matches!(bucket, Bucket::Opponent | Bucket::Uncertain)
113         {
114             out.created_exposed += value;
115         }
116         if is_own_half(tree, plan, &opp_d) && !matches!(bucket, Bucket::Ours) {
117             out.own_half_exposed += value;
118         }
119     }
120
121     out
122 }
123
124 // — Pressure governor (Task 0/1: score contract + live exposure to planning) —————
125
126 /// Green < Yellow < Orange < Red (declaration order = derived `Ord`): the farm-pressure
127 /// ladder from docs/pressure-aware-farm.md. Escalation is purely a function of the
128 /// OBSERVED `Ownership` buckets below – never turn number alone (static turn-only gates
129 /// are a proven dead end: earlyroam boss 0/8, lateseedhome -1.2).
130 #[derive(Clone, Copy, Debug, PartialEq, Eq, PartialOrd, Ord)]
131 pub enum PressureState {
132     Green,
133     Yellow,
134     Orange,
135     Red,
136 }
137
138 impl Default for PressureState {
139     fn default() -> Self {
140         PressureState::Green
141     }
142 }
143
144 /// Compact pressure result consumed by `tactics::Plan` / `planner.rs`. `exposed_created_cells`
145 /// and `released_seed_cells` are POSITION sets (not iterated for ordering – only ever
146 /// `.contains()`-checked by callers, so HashSet's unspecified iteration order cannot leak
147 /// into emitted command order; see the determinism note on `assess` below).
148 #[derive(Clone, Debug, Default, PartialEq, Eq)]
149 pub struct Pressure {
150     pub own_half_exposed: i32,
151     pub created_exposed: i32,
152     pub pressure_score: i32,
153     pub state: PressureState,
154     /// Created/local farm trees (is_created_farm_tree) classified Opponent or Uncertain –
155     /// "not safely ours". Drives Task 2 Step 3's liquidation-priority bonus.
156     pub exposed_created_cells: HashSet<Cell>,
157     /// Subset of `plan.seed_cells` that pressure has released from protection (Task 2 Step
158     /// 2). Deliberately STRICTER than `exposed_created_cells`: only a seed tree that is
159     /// ITSELF definitively opponent-bound (bucket == Opponent, not merely Uncertain)
160     /// releases, and only once the aggregate state has escalated to Orange/Red – seed
161     /// supply is the most dangerous lever in this codebase's history (arena deforestation
162     /// stalls when the farm's seed source dies), so releasing it is conservative by design.
163     pub released_seed_cells: HashSet<Cell>,
164 }
165
166 fn classify_pressure(own_half_exposed: i32, created_exposed: i32, definite_opponent: bool) -> PressureState {
167     if created_exposed > 0 {
168         if definite_opponent {
169             PressureState::Red
170         } else {

```

```

171         PressureState::Orange
172     }
173 } else if own_half_exposed > 0 {
174     PressureState::Yellow
175 } else {
176     PressureState::Green
177 }
178 }
179
180 /// Computed ONCE per turn by `tactics::plan_impl` (never inside planner.rs's per-troll
181 /// `candidates()` hot loop – Task 1 Step 2). Calls the UNCHANGED `analyze` once, then a
182 /// second pass bounded by created-farm-tree count (not map size) to classify exposure and
183 /// find the Red trigger ("opponent ETA makes preserving nearby farm value worse than
184 /// conversion" == at least one created-exposed tree is DEFINITELY opponent-bound, i.e.
185 /// `Bucket::Opponent`, not just a close `Bucket::Uncertain` race).
186 ///
187 /// Determinism: iterates `state.trees` (a `Vec`, stable order already) and inserts into
188 /// HashSets that are only ever `.contains()`-queried afterward (never iterated for ordered
189 /// output) – so HashSet's unspecified internal order cannot affect any emitted command.
190 pub fn assess(state: &State, plan: &Plan) -> Pressure {
191     let own = analyze(state, plan);
192
193     let mut exposed_created_cells: HashSet<Cell> = HashSet::new();
194     let mut definite_opponent = false;
195     for tree in &state.trees {
196         if tree_value(tree) <= 0 || !is_created_farm_tree(tree, plan) {
197             continue;
198         }
199         match classify_tree(state, tree) {
200             Bucket::Opponent => {
201                 exposed_created_cells.insert(tree.pos());
202                 definite_opponent = true;
203             }
204             Bucket::Uncertain => {
205                 exposed_created_cells.insert(tree.pos());
206             }
207             Bucket::Ours | Bucket::Dead => {}
208         }
209     }
210
211     let pressure_score = own.own_half_exposed + own.created_exposed;
212     let state_level = classify_pressure(own.own_half_exposed, own.created_exposed, definite_opponent);
213
214     /// Task 2 Step 2 (seed-reserve release): conservative on purpose – see the doc comment
215 /// on `Pressure::released_seed_cells`. Gated on the AGGREGATE state (Orange/Red, the
216 /// plan's literal wording) as a belt-and-suspenders sanity check, even though in
217 /// practice a seed tree with bucket==Opponent already forces state_level to Red on its
218 /// own (a seed tree is always a created-farm tree).
219     let released_seed_cells: HashSet<Cell> = if state_level >= PressureState::Orange {
220         plan.seed_cells
221             .iter()
222             .filter(|&&pos| {
223                 state
224                     .trees
225                     .iter()
226                     .find(|t| t.pos() == pos)
227                     .map_or(false, |t| classify_tree(state, t) == Bucket::Opponent)
228             })
229             .copied()
230             .collect()
231     } else {
232         HashSet::new()
233     };
234
235     Pressure {
236         own_half_exposed: own.own_half_exposed,
237         created_exposed: own.created_exposed,
238         pressure_score,
239         state: state_level,
240         exposed_created_cells,
241         released_seed_cells,
242     }
243 }
244
245 /// DEBUG telemetry (Task 1 Step 4): @TFPRESSCFG once at turn 1 (constants, near the farm
246 /// constants per the plan), then @TFPRESS at the same cadence as @TFOWN. Reads Plan's
247 /// ALREADY-computed `pressure` field – no recomputation, unlike `log` above (which still
248 /// calls `analyze` fresh; the two numbers agree because nothing mutates state/plan between
249 /// `tactics::plan` returning and this DEBUG print).
250 pub fn log_pressure(state: &State, plan: &Plan) {
251     if state.turn == 1 {
252         PRESS_CFG_PRINTED.with(|printed| {
253             let mut printed = printed.borrow_mut();
254             if !*printed {
255                 eprintln!("@TFPRESSCFG farm_floor={}", GE_PRESSURE_FARM_FLOOR);
256                 *printed = true;
257             }
258         });
259     }
260 }

```

```

259     }
260     if !should_emit(state.turn) {
261         return;
262     }
263     eprintln!(
264         "@TFPRESS t={ } own_half_exposed={ } created_exposed={ } pressure_score={ } state={:?} exposed_n={ } released_n={ }",
265         state.turn,
266         plan.pressure.own_half_exposed,
267         plan.pressure.created_exposed,
268         plan.pressure.pressure_score,
269         plan.pressure.state,
270         plan.pressure.exposed_created_cells.len(),
271         plan.pressure.released_seed_cells.len(),
272     );
273 }
274
275 fn ensure_initial(state: &State) {
276     INITIAL_READY.with(|ready| {
277         if *ready.borrow() {
278             return;
279         }
280         INITIAL_TREES.with(|s| {
281             let mut s = s.borrow_mut();
282             s.clear();
283             for tree in &state.trees {
284                 s.insert((tree.pos(), tree.tree_type.clone()));
285             }
286         });
287         *ready.borrow_mut() = true;
288     });
289 }
290
291 fn should_emit(turn: i32) -> bool {
292     turn == 75 || turn == 150 || turn == 225 || turn == TOTAL_TURNS || turn % 5 == 0
293 }
294
295 fn tree_value(tree: &Tree) -> i32 {
296     let wood = 4 * tree.size.max(0);
297     let fruit = tree.fruits.max(0);
298     let future = if tree.fruits > 0 && (tree.tree_type == "BANANA" || tree.tree_type == "APPLE") {
299         OWN_FUTURE_SEED_VALUE
300     } else {
301         0
302     };
303     wood + fruit + future
304 }
305
306 fn classify_tree(state: &State, tree: &Tree) -> Bucket {
307     let my_eta = best_side_eta(state, &state.my_trolls, state.my_shack, tree);
308     let opp_eta = best_side_eta(state, &state.opp_trolls, state.opp_shack, tree);
309     let turns_rem = TOTAL_TURNS - state.turn + 1;
310
311     if my_eta > turns_rem && opp_eta > turns_rem {
312         return Bucket::Dead;
313     }
314     if my_eta + OWN_MARGIN_TURNS <= opp_eta {
315         return Bucket::Ours;
316     }
317     if opp_eta + OWN_MARGIN_TURNS <= my_eta {
318         return Bucket::Opponent;
319     }
320     Bucket::Uncertain
321 }
322
323 fn best_side_eta(state: &State, workers: &[Troll], shack: Cell, tree: &Tree) -> i32 {
324     let bank_cells = bank_cells(state, shack);
325     if bank_cells.is_empty() {
326         return INF;
327     }
328     let tree_d = bfs_distances(&state.walkable, &[tree.pos()]);
329     let tree_to_bank = min_dist(&tree_d, &bank_cells);
330     if tree_to_bank >= INF {
331         return INF;
332     }
333
334     let mut best = INF;
335     for worker in workers {
336         let ms = worker.movement_speed.max(1);
337         let from_worker = bfs_distances(&state.walkable, &[worker.pos()]);
338         let move_dist = from_worker.get(&tree.pos()).copied().unwrap_or(INF);
339         if move_dist >= INF {
340             continue;
341         }
342         let prebank = prebank_turns(worker, &from_worker, &bank_cells);
343         let move_turns = div_ceil(move_dist, ms);
344         let bank_turns = div_ceil(tree_to_bank, ms) + 1;
345
346         if tree.size > 0 && worker.chop_power > 0 {

```

```

347         let action_turns = div_ceil(tree.health.max(1), worker.chop_power.max(1));
348         best = best.min(prebank + move_turns + action_turns + bank_turns);
349     }
350     if tree.fruits > 0 && worker.harvest_power > 0 {
351         best = best.min(prebank + move_turns + 1 + bank_turns);
352     }
353 }
354 best
355 }
356
357 fn prebank_turns(
358     worker: &Troll,
359     from_worker: &std::collections::HashMap<Cell, i32>,
360     bank_cells: &[Cell],
361 ) -> i32 {
362     if worker.free_capacity() > 0 {
363         return 0;
364     }
365     let d = min_dist(from_worker, bank_cells);
366     if d >= INF {
367         INF
368     } else if d == 0 || is_any_near(worker.pos(), bank_cells) {
369         1
370     } else {
371         div_ceil(d, worker.movement_speed.max(1)) + 1
372     }
373 }
374
375 fn bank_cells(state: &State, shack: Cell) -> Vec<Cell> {
376     let mut out: Vec<Cell> = ortho_neighbors(shack)
377         .iter()
378         .copied()
379         .filter(|c| state.walkable.contains(c))
380         .collect();
381     if state.walkable.contains(&shack) {
382         out.push(shack);
383     }
384     out
385 }
386
387 fn min_dist(d: &std::collections::HashMap<Cell, i32>, cells: &[Cell]) -> i32 {
388     cells
389         .iter()
390         .filter_map(|c| d.get(c).copied())
391         .min()
392         .unwrap_or(INF)
393 }
394
395 fn is_any_near(cell: Cell, cells: &[Cell]) -> bool {
396     cells.iter().any(|&c| manhattan(cell, c) <= 1)
397 }
398
399 fn div_ceil(n: i32, d: i32) -> i32 {
400     if n <= 0 {
401         0
402     } else {
403         (n + d - 1) / d.max(1)
404     }
405 }
406
407 fn is_created_farm_tree(tree: &Tree, plan: &Plan) -> bool {
408     if tree.tree_type != "BANANA" {
409         return false;
410     }
411     let in_farm = plan
412         .farm_d
413         .get(&tree.pos())
414         .map_or(false, |&d| d <= plan.farm_r);
415     let near_tent = manhattan(tree.pos(), plan.shack) <= OWN_CREATED_NEAR_TENT_R;
416     if !in_farm && !near_tent {
417         return false;
418     }
419     INITIAL_TREES.with(|s| !s.borrow().contains(&(tree.pos(), tree.tree_type.clone())))
420 }
421
422 fn is_own_half(tree: &Tree, plan: &Plan, opp_d: &std::collections::HashMap<Cell, i32>) -> bool {
423     let my = plan.farm_d.get(&tree.pos()).copied().unwrap_or(INF);
424     let opp = opp_d.get(&tree.pos()).copied().unwrap_or(INF);
425     my <= opp
426 }

```